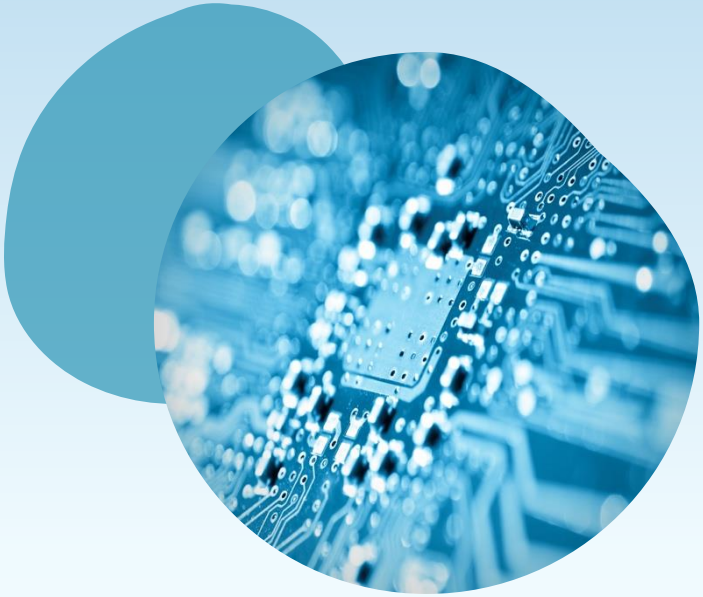
Analog to Digital Converter (ADC)



Embedded Software for the Internet Of Things

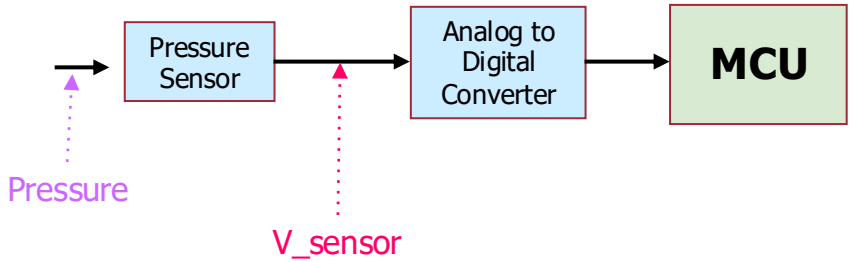
Kasim Sinan Yildirim

Analog to Digital Conversion

- Embedded systems often need to measure values of physical phoneme
- Sensor examples:
  - Temperature, Light (or infrared or ultraviolet) intensity, Pressure, Blood pressure, Acceleration
- These parameters are usually continuous (**analog**)
- Computers can process data in a discrete (**digital**) form

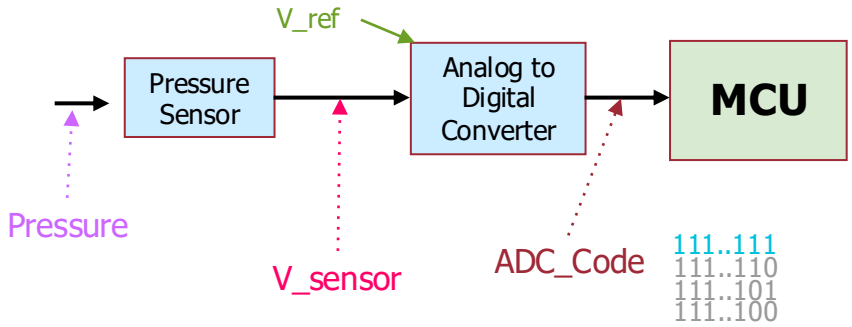
Example Analog Sensor

- We calculate how deep underwater an object is:
  - Sensor detects **pressure** and generates a **proportional** output voltage V_sensor



Example Analog Sensor

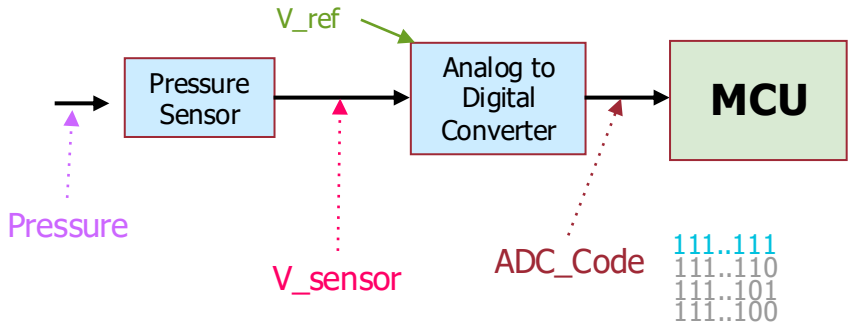
- We calculate how deep underwater an object is:
  - Sensor detects **pressure** and generates a **proportional** output voltage **V_sensor**
  - **Analog to Digital Converter** generates a proportional digital integer (code) based on **V_sensor** and **V_ref**

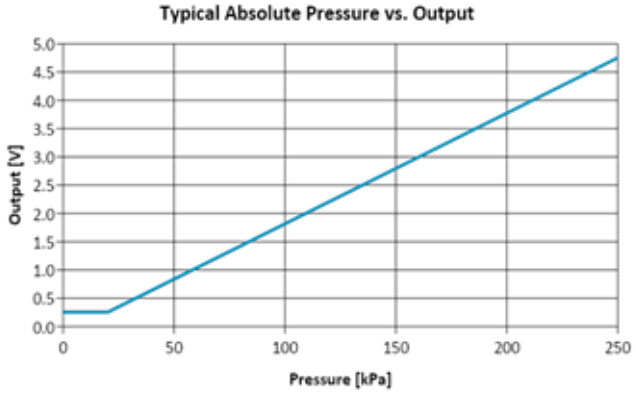


Example Analog Sensor

Code can convert that **ADC_Code (integer)** to a something more useful
- **V_sensor**: representing the voltage,
- **Pressure_kPa**: representing the pressure
- **Depth_ft**: representing the depth

- We calculate how deep underwater an object is:
  - Sensor detects **pressure** and generates a **proportional** output voltage V_sensor
  - **Analog to Digital Converter** generates a proportional digital integer (code) based on **V_sensor** and **V_ref**



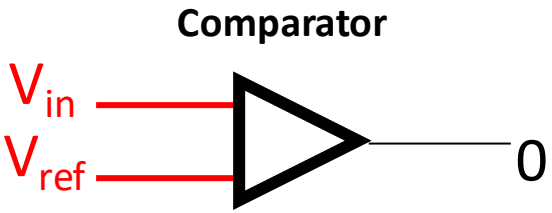


Software on MCU

```
ADC_Code = adc_read();
V_sensor = ADC_code * V_ref / ADC_MASK;
Pressure_kPa = 250 * (V_sensor / V_supply + 0.04);
Depth_ft = 33 * (Pressure_kPa - Atmos_Press_kPa) / 101.3;
```

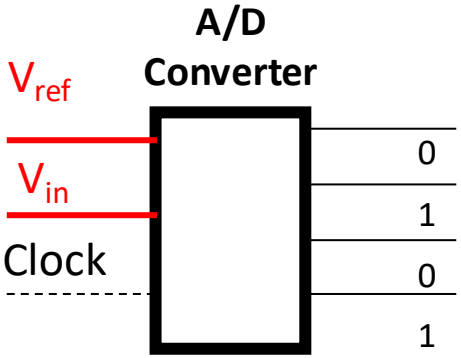
Getting From Analog to Digital

- A Comparator tells us "**Is Vin > Vref?**"
  - Compares an analog input voltage with an analog reference voltage and determines which is larger, returning a 1-bit number
    - E.g. Indicate if depth > 100ft
  - Set Vref to voltage pressure sensor returns with 100 ft depth.



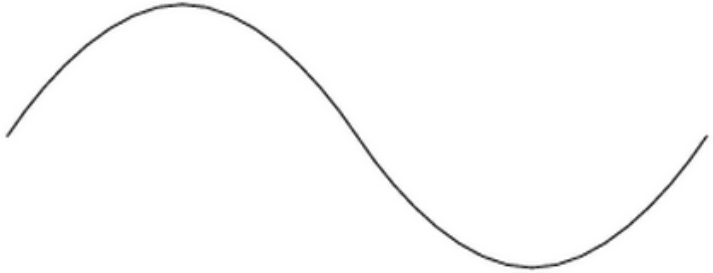
Getting From Analog to Digital

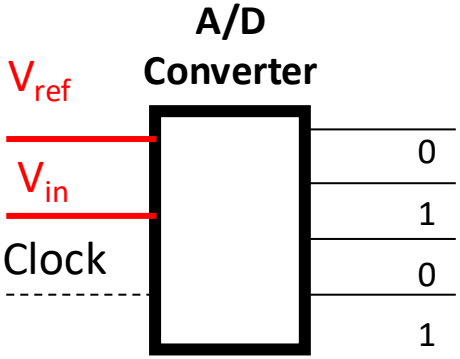
- An **Analog to Digital converter (ADC)** tells us how large Vin is as a fraction of Vref.
  - Reads an analog input signal (usually a voltage) and produces a corresponding **multi-bit number** at the output.
    - E.g. calculate the depth



Getting From Analog to Digital

- The ADC observes a continuous signal at discrete times and represents each observation with a finite code.

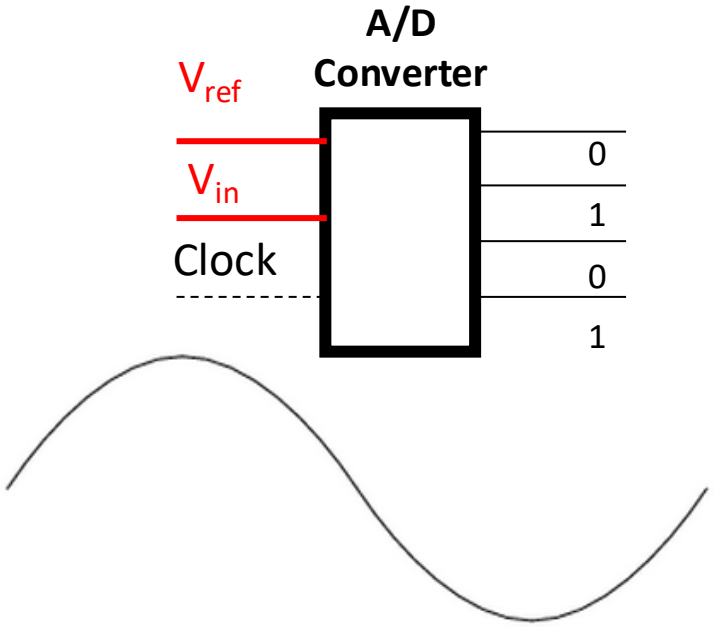




$$f_s = 1 / T_s$$

Sampling in time

- The **sample period** is the time between observations
- The **sample rate** is the reciprocal of the sample period
- A **sample-and-hold circuit** keeps the input stable during conversion
- **Timing jitter** changes the effective sampling instant



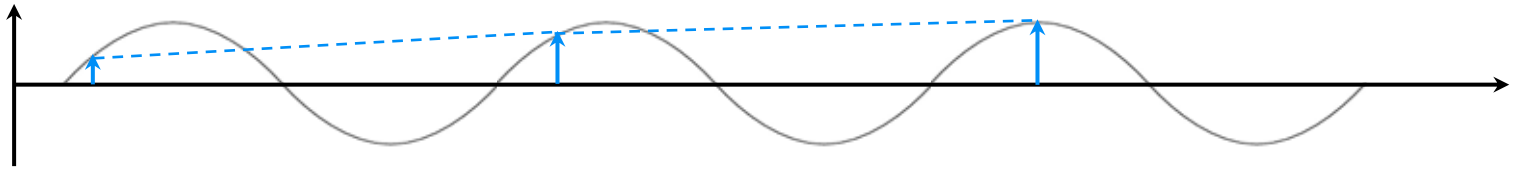
$$f_s = 1 / T_s$$

Sampling Frequency

- The sampling rate has to be high enough to reproduce the original signal.
- **Shannon-Nyquist sampling theorem**

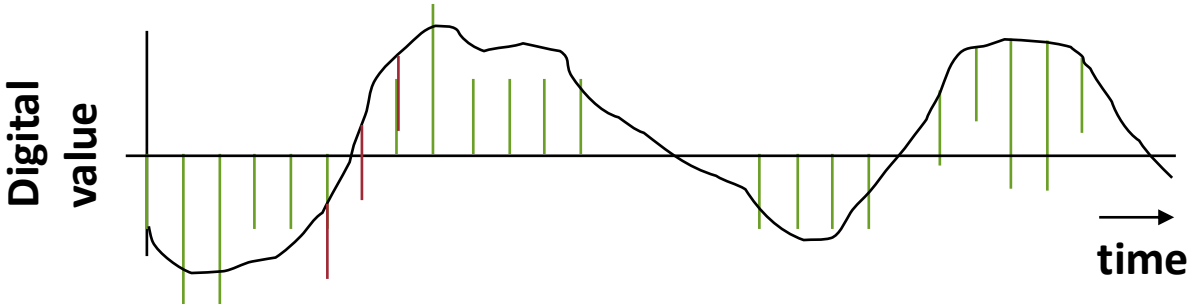
$$F_{sample} >= 2*F_{max}$$ frequency component

- Frequency components above ½ F_{sample} are **aliased**
  - distorting the measured signal



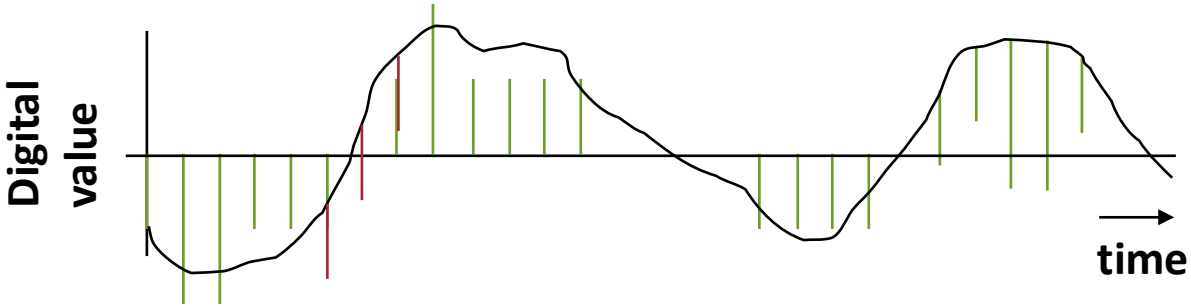
Waveform Sampling and Quantization

- Each such sample represents the instantaneous amplitude at the instant of sampling
  - "At 37 ms, the input is **1.9134191451345145123431l… V**"
  - Sampling converts a continuous time signal to a **discrete time signal**



Waveform Sampling and Quantization

- The sample can now be **quantized** into a digital value
  - Quantization represents a continuous (**analog**) value with the closest discrete (**digital**) value
  - "The sampled input voltage of **1.9134191451345145123431 1… V** is best represented by the code **0x018**:
- 0x018 represents **the range** of 1.901 V to 1.9980V



Resolution and the ideal ADC code

- **Least Significant Bit (LSB):** the smallest change in input voltage that causes a change in the converter's digital output code.

$$\mathrm{LSB} = \frac{V_{in} - V_{min}}{2^N}$$

$$\mathbf{code} = \left\lfloor \frac{V_{in} - V_{min}}{V_{max} - V_{min}} 2^N \right\rfloor$$

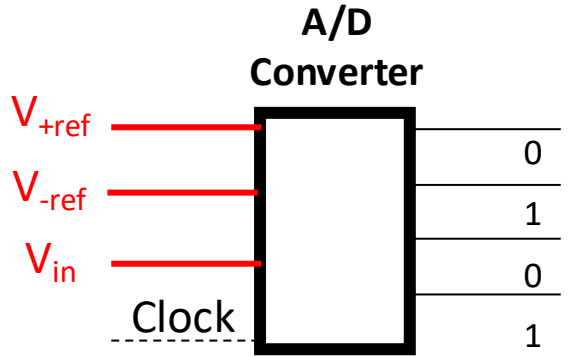
floor(X) ***nearest integer*** *I such that I<=X*

Example	Value
Resolution	12 bits, 4096 nominal levels
Input range	0 to 3.3 V
Ideal LSB	3.3 V / 4096 ≈ 0.806 mV
Code for 1.65 V	approximately 2048

Resolution and the ADC code

- $\mathbf{V_{in}}$ = sampled input voltage
- $\mathbf{V_{+ref}}$ = upper voltage reference
- $\mathbf{V_{-ref}}$ = lower voltage reference
- **N** = number of bits of resolution in ADC

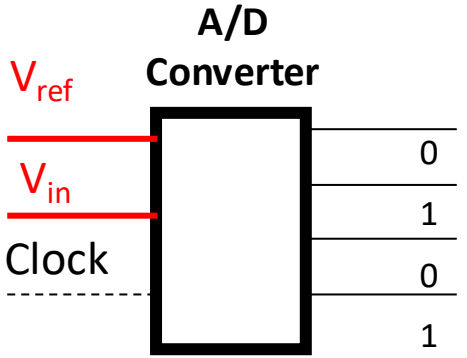
$$\mathbf{code} = \left\lfloor \frac{V_{in} - V_{-ref}}{V_{+ref} - V_{-ref}} 2^{N} \right\rfloor$$



Resolution and the ADC code

- $\mathbf{V_{in}}$ = sampled input voltage
- $\mathbf{V_{+ref}}$ = upper voltage reference
- $V_{-ref} = 0$
- **N** = number of bits of resolution in ADC

$$\mathbf{code} = \left\lfloor \frac{V_{in}}{V_{+ref}} 2^{N} \right\rfloor$$

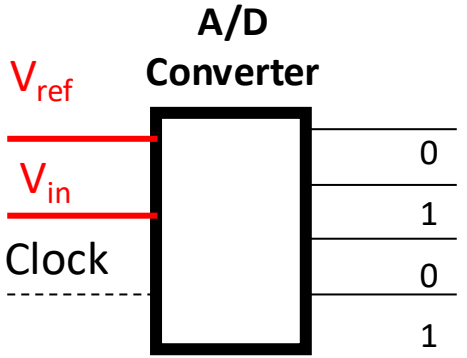


Resolution and the ADC code

- V_{in} = sampled input voltage
- V_{+ref} = upper voltage reference
- $V_{-ref} = 0$
- **N** = number of bits of resolution in ADC

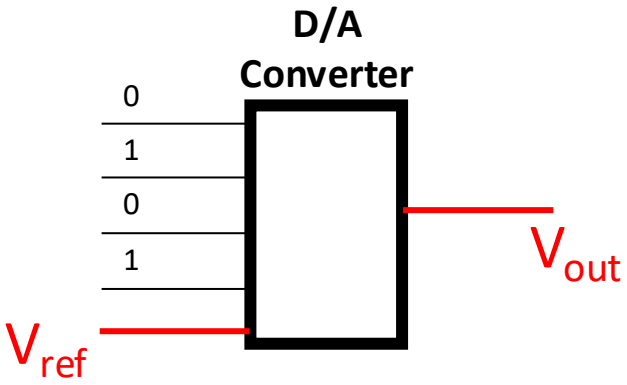
$$\mathbf{code} = \left\lfloor \frac{V_{in}}{V_{+ref}} 2^{N} \right\rfloor$$

$$\mathbf{code} = 388 = \left\lfloor \frac{3.3V}{5V} 2^{10} \right\rfloor$$



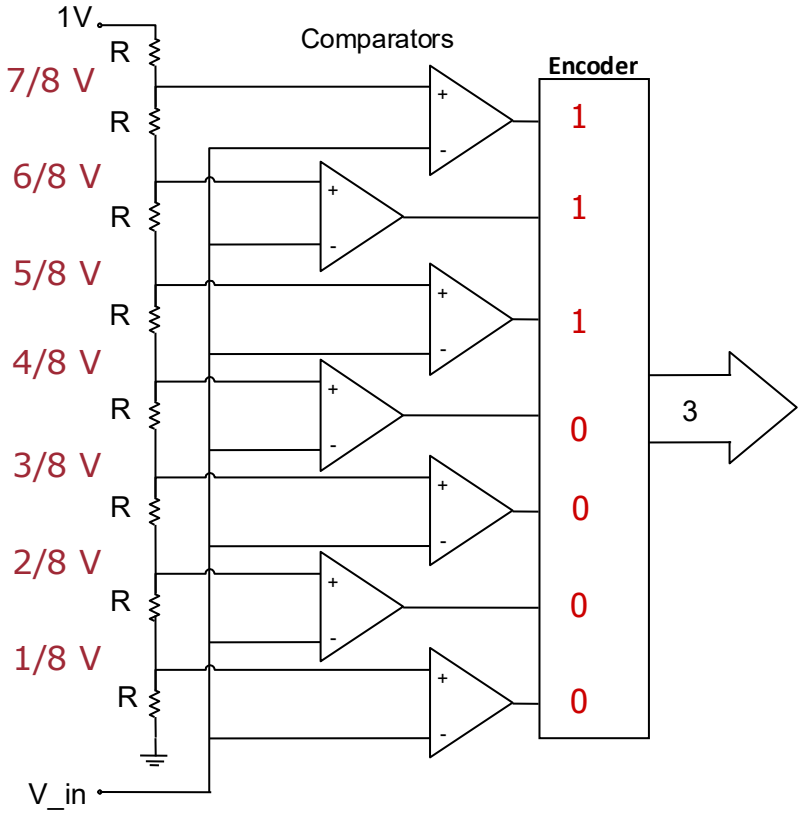
Digital to Analog Conversion

- DAC: “Generate the analog voltage which is this **fraction** of V_{ref}”



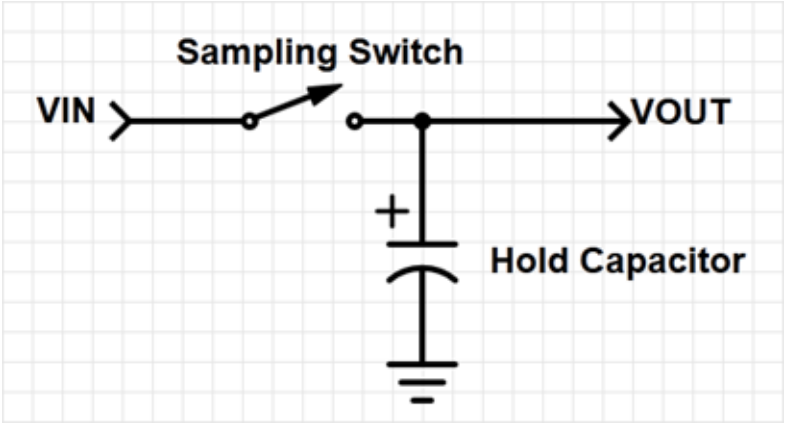
Flash Converter Circuit

- A multi-level voltage divider is used to set voltage levels over the complete range of conversion.
  - A comparator is used at each level to determine whether the voltage is lower or higher than the level.
- The series of comparator outputs are encoded to a binary number in digital logic (a priority encoder)



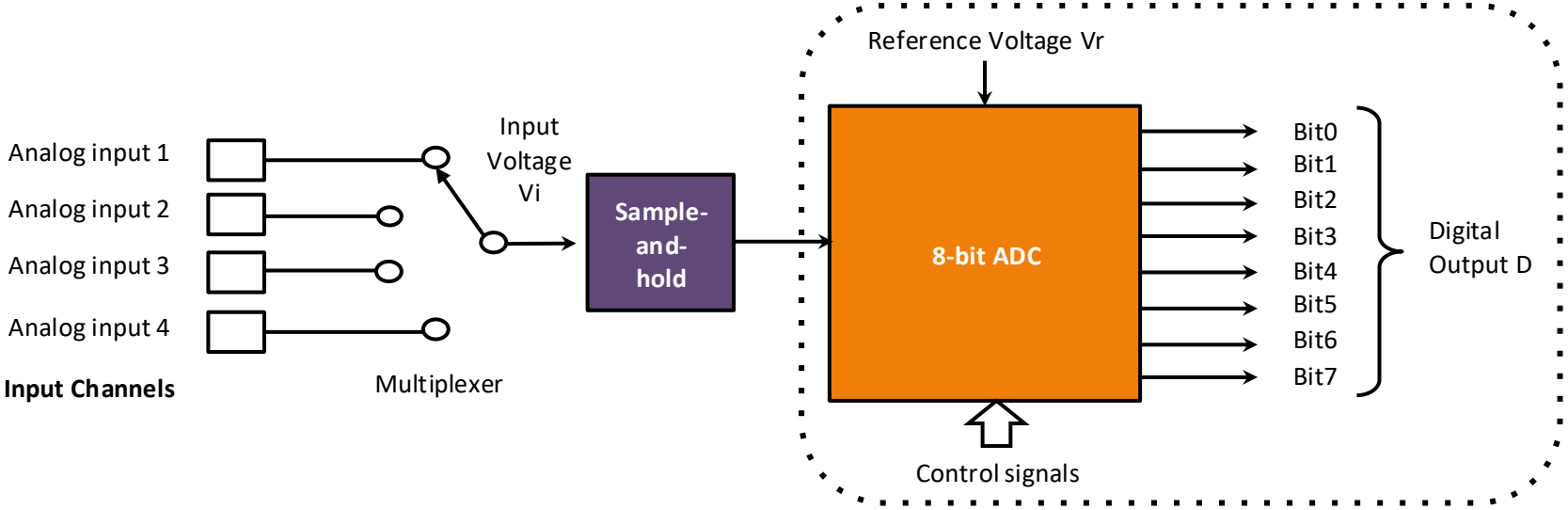
Sample and Hold Devices

- The input analog signal needs to be held **constant** during conversion
  - This function is accomplished by a sample and hold device
  - The capacitor **'holds'** the input voltage until a conversion has been completed.



Analog-Digital-Conversion-System

- An ADC-System usually consists of three stages:
  - **Multiplexer**: reduces the number of required ADC components
  - **Sample-and-hold**: keeps the analog signal during the conversion constant.
    - Time discretization!
  - **ADC**: converts the analog into digital signal

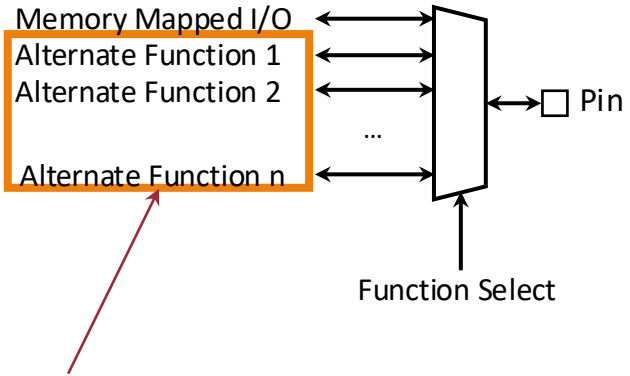


Some Performance Metrics

- **Number of bits** determines overall accuracy.
  - There are always limitations and inaccuracies present in the conversion from Analog to Digital.
- Conversion time: between start of conversion and generation of result.
  - Conversion rate = inverse of conversion time.

GPIO Alternative Functions

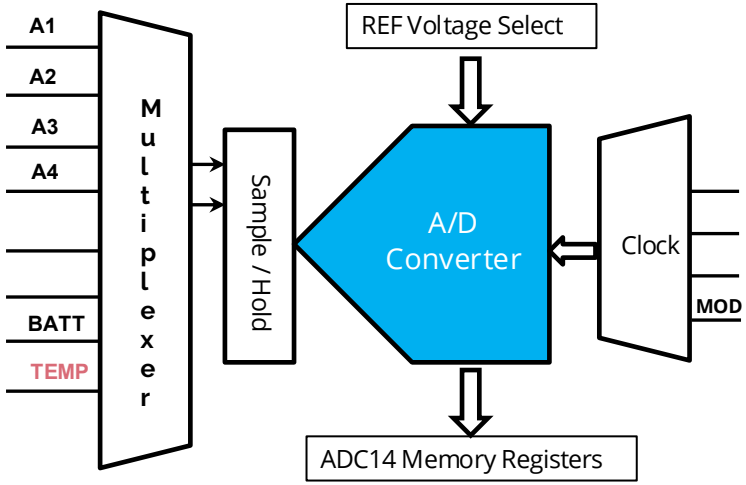
- Pins may have different features:
  - may also have analog paths for ADC / DAC etc.
- To enable an alternative function, configure the appropriate register!



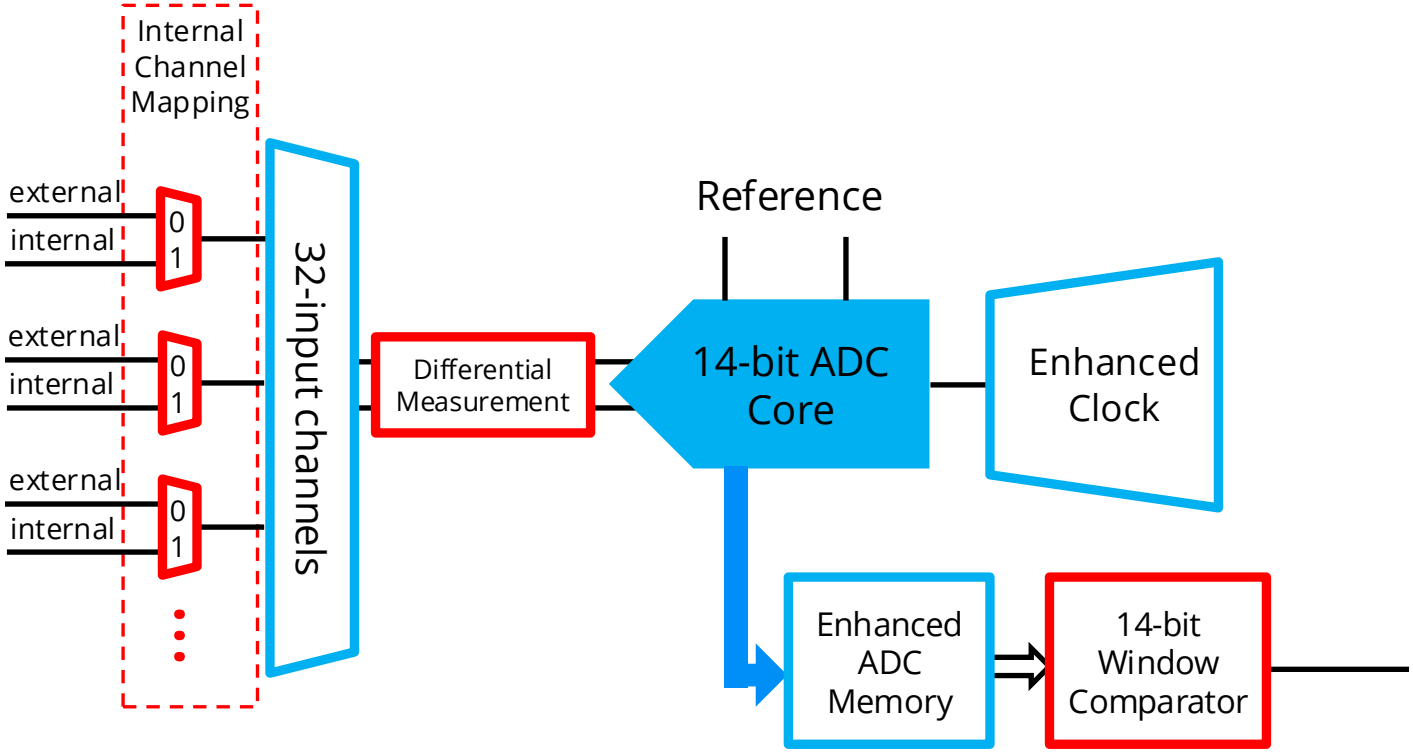
Select appropriate analog functionality

MSP432 ADC Overview

- Analog to Digital Converter
  - **14-bit Accuracy**
  - **32-input channels**
  - Separate internal channel for **TempSensor**

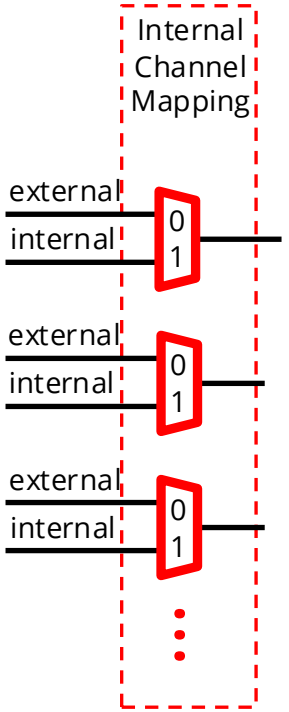


MSP432 ADC Overview



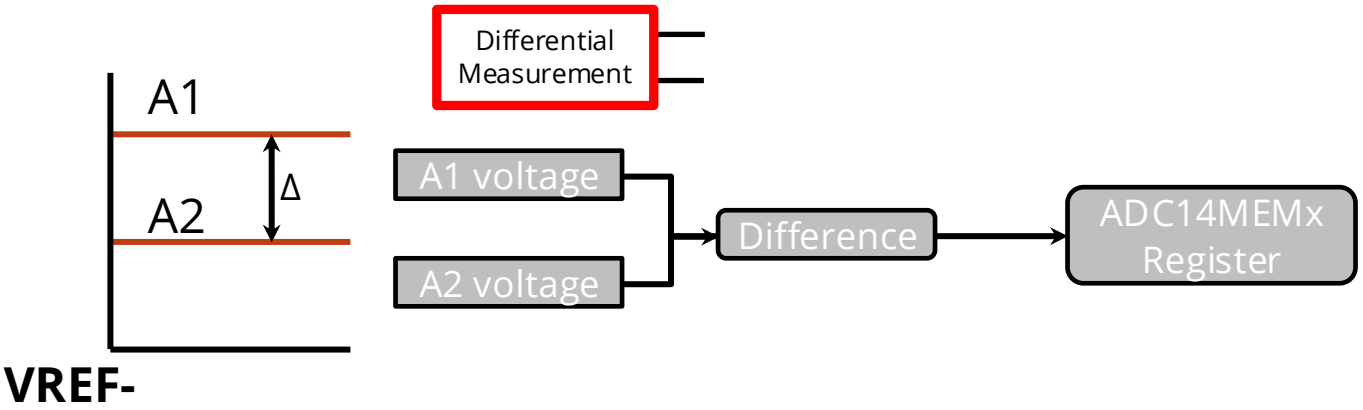
Channels

- A26-31 can map to either an External Input or Internal ADC input
  - Internal inputs include the **temperature sensor** and **battery monitor**.



Differential Measurement

- Combine input channels to create differential input channels
  - ADC will measure the **difference** between two channels and store this value in the ADC14MEMx register

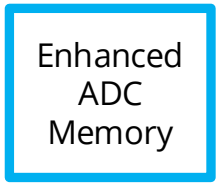


ADC Clock and Memory

- Clock sources, including:
  - MODOSC = 25MHz
  - SYSOSC = 5MHz

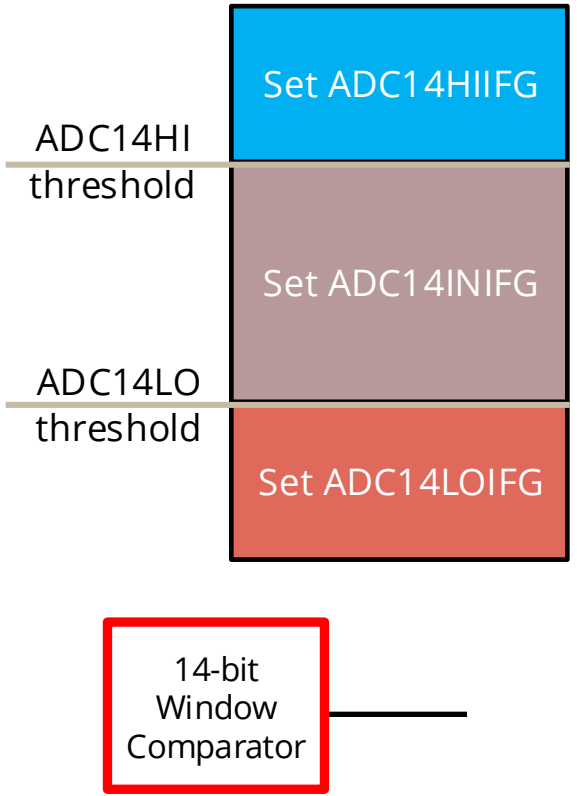


- 32 ADC14MEMx registers

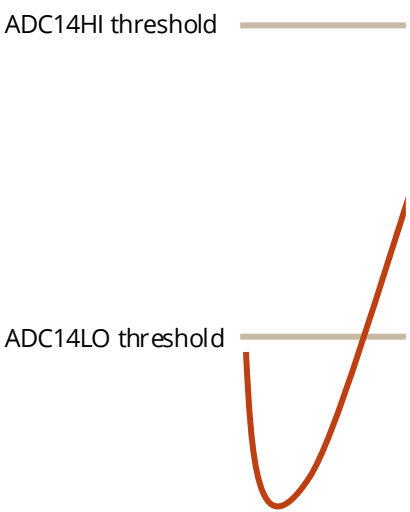


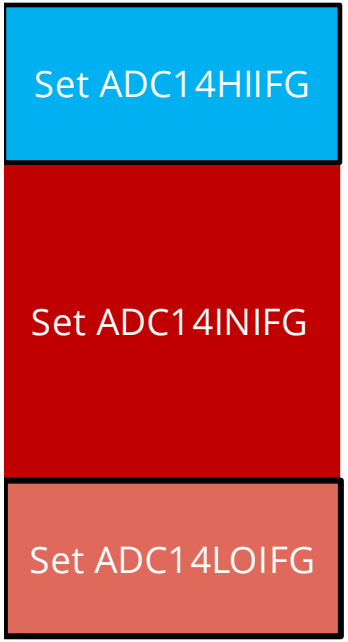
MSP432 ADC Overview

- Configurable input threshold levels
- Conversion results automatically compared against the thresholds
- Hi, Lo, and In interrupts indicate which range the result falls in
  - Same thresholds shared among all channels
  - Useful for low power because device can stay in sleep mode until result falls in window

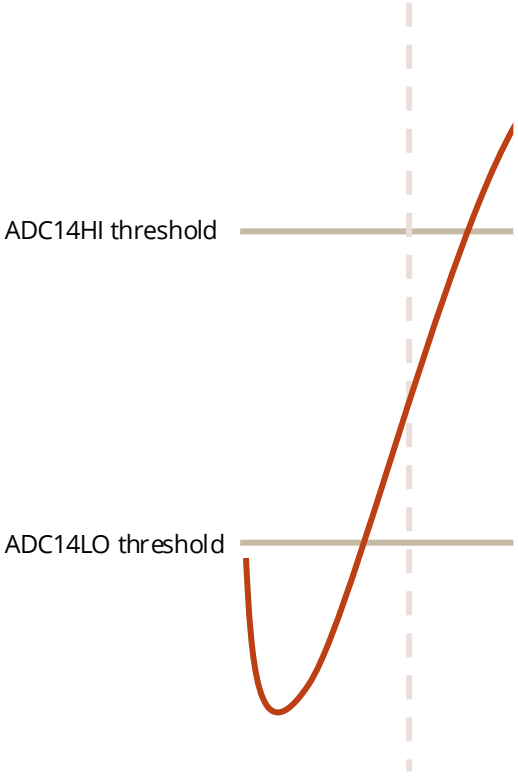


MSP432 ADC Overview



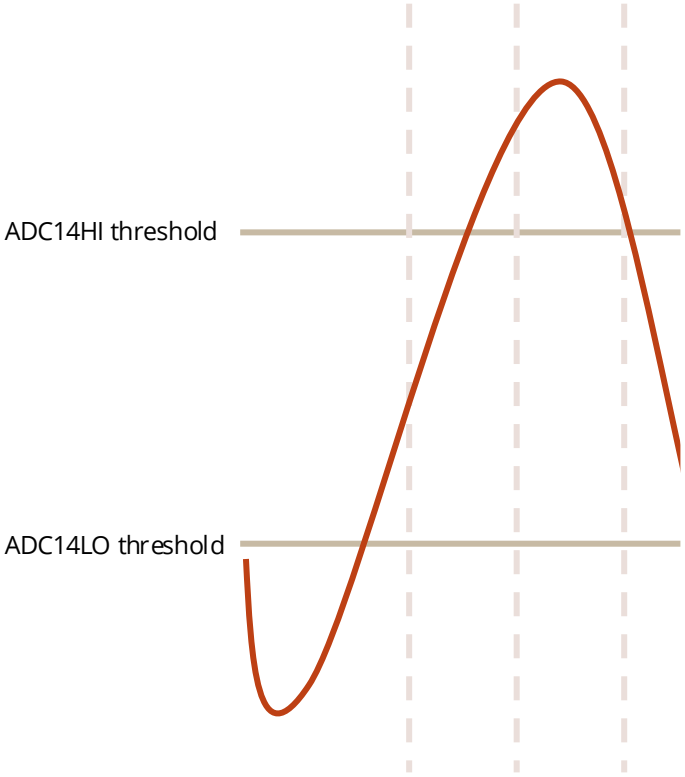


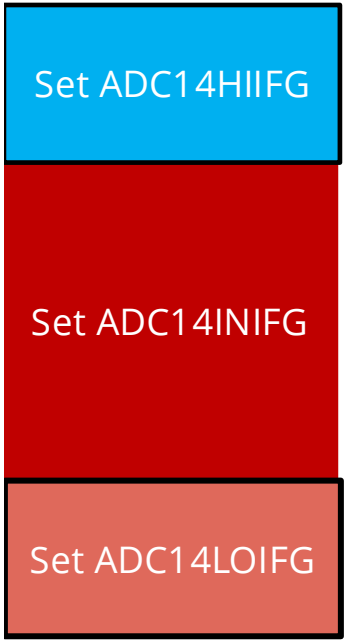
MSP432 ADC Overview



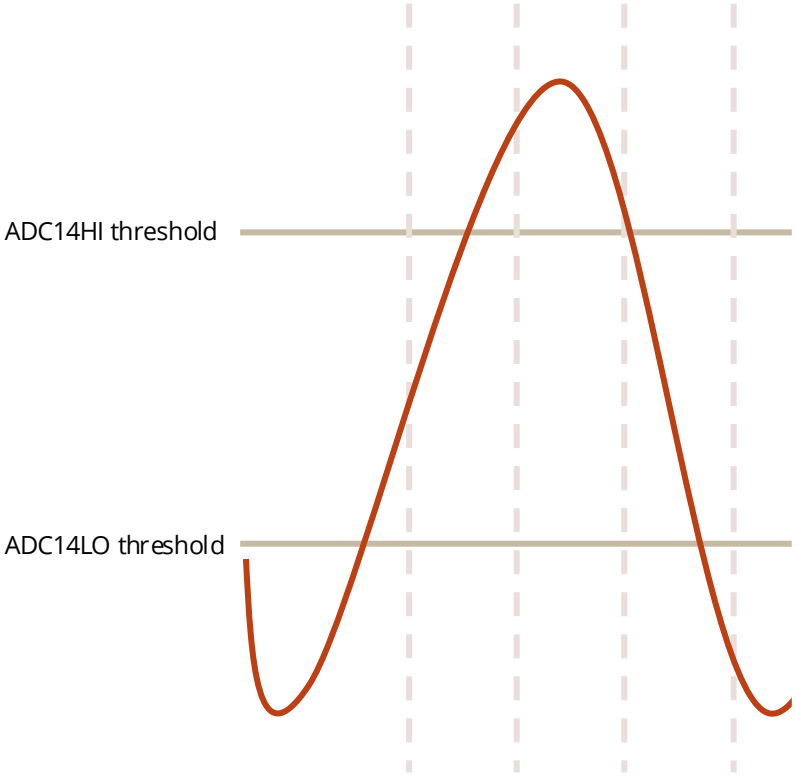


MSP432 ADC Overview



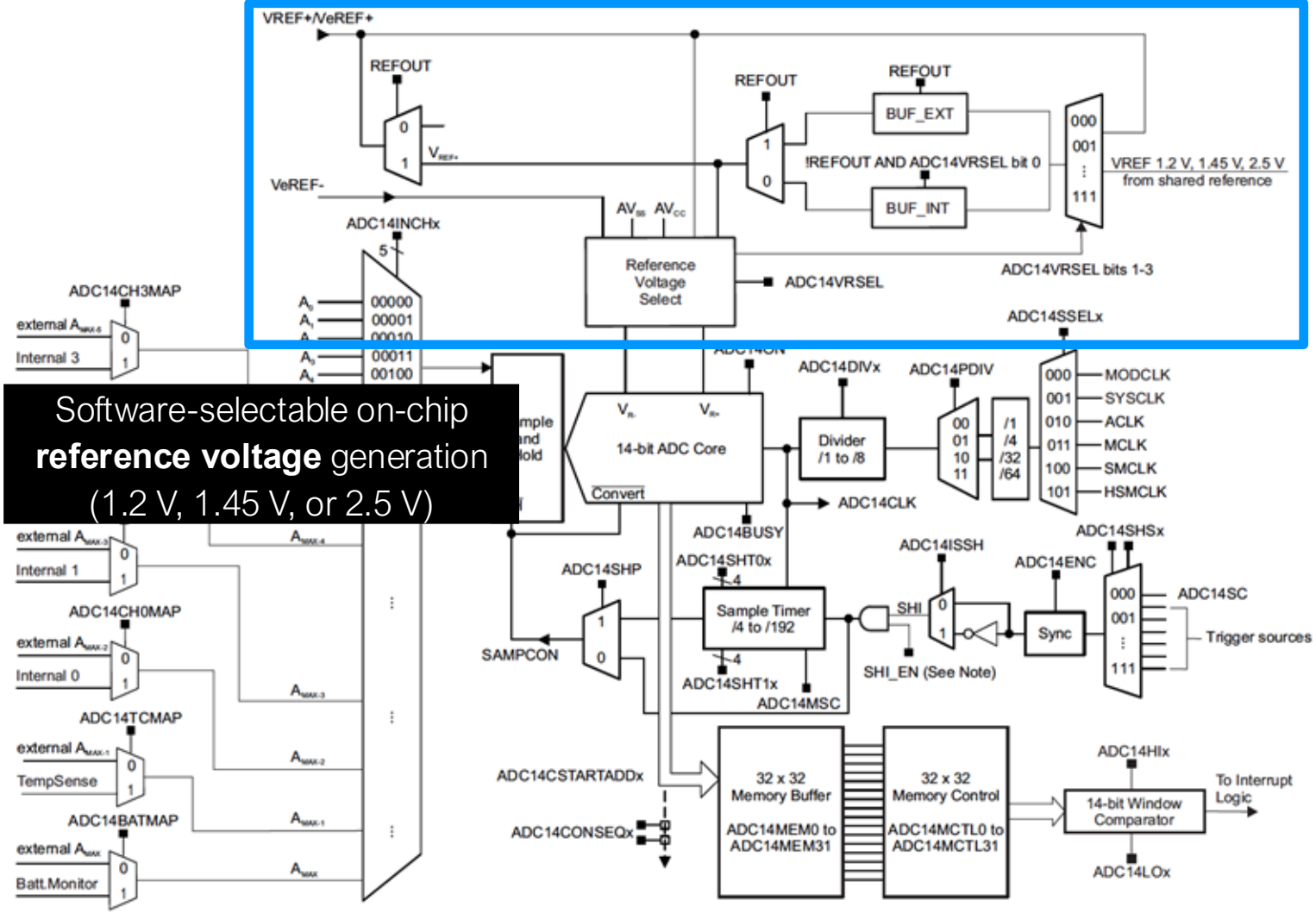


MSP432 ADC Overview



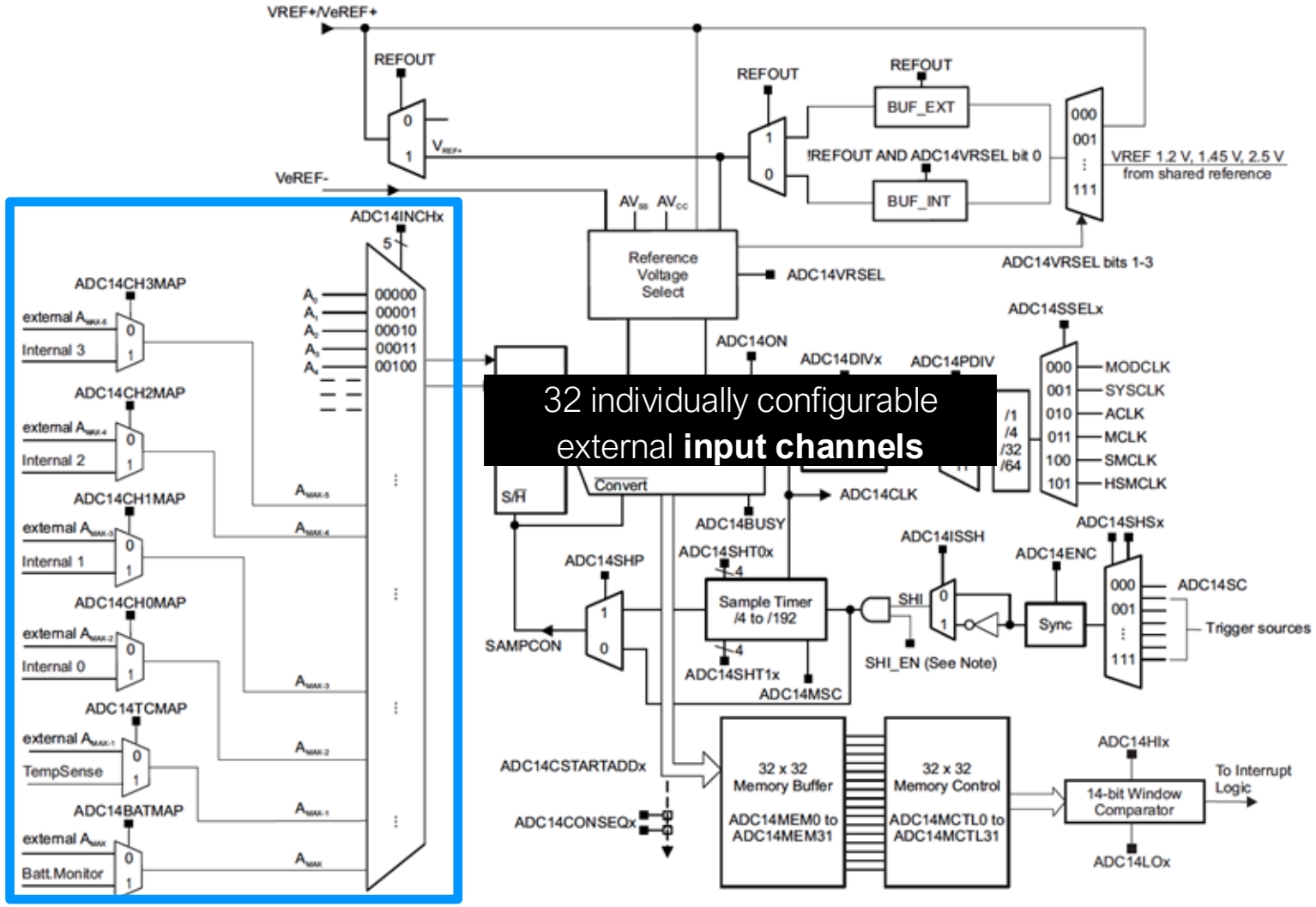


MSP432 ADC

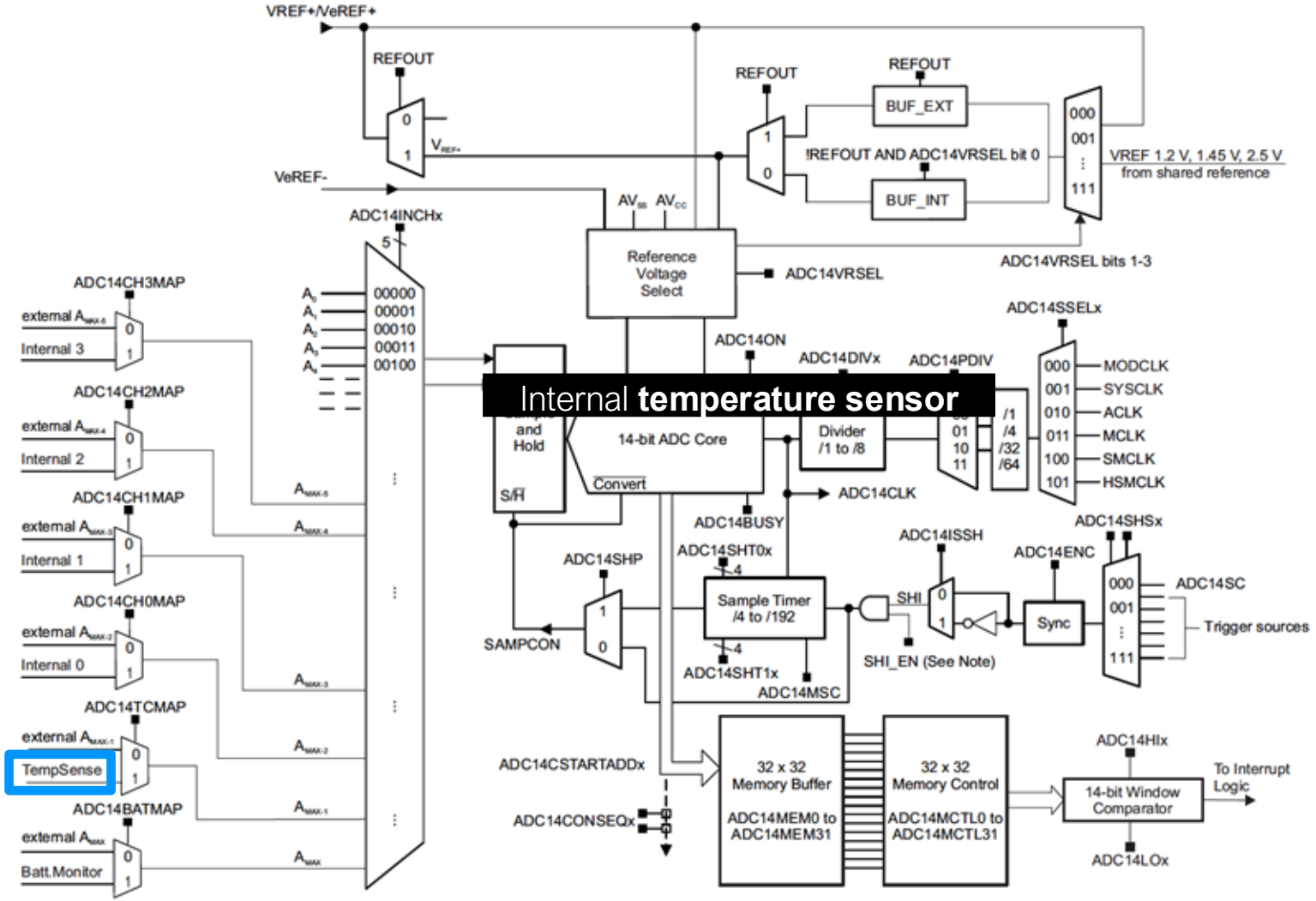


Software-selectable on-chip **reference voltage** generation (1.2 V, 1.45 V, or 2.5 V)

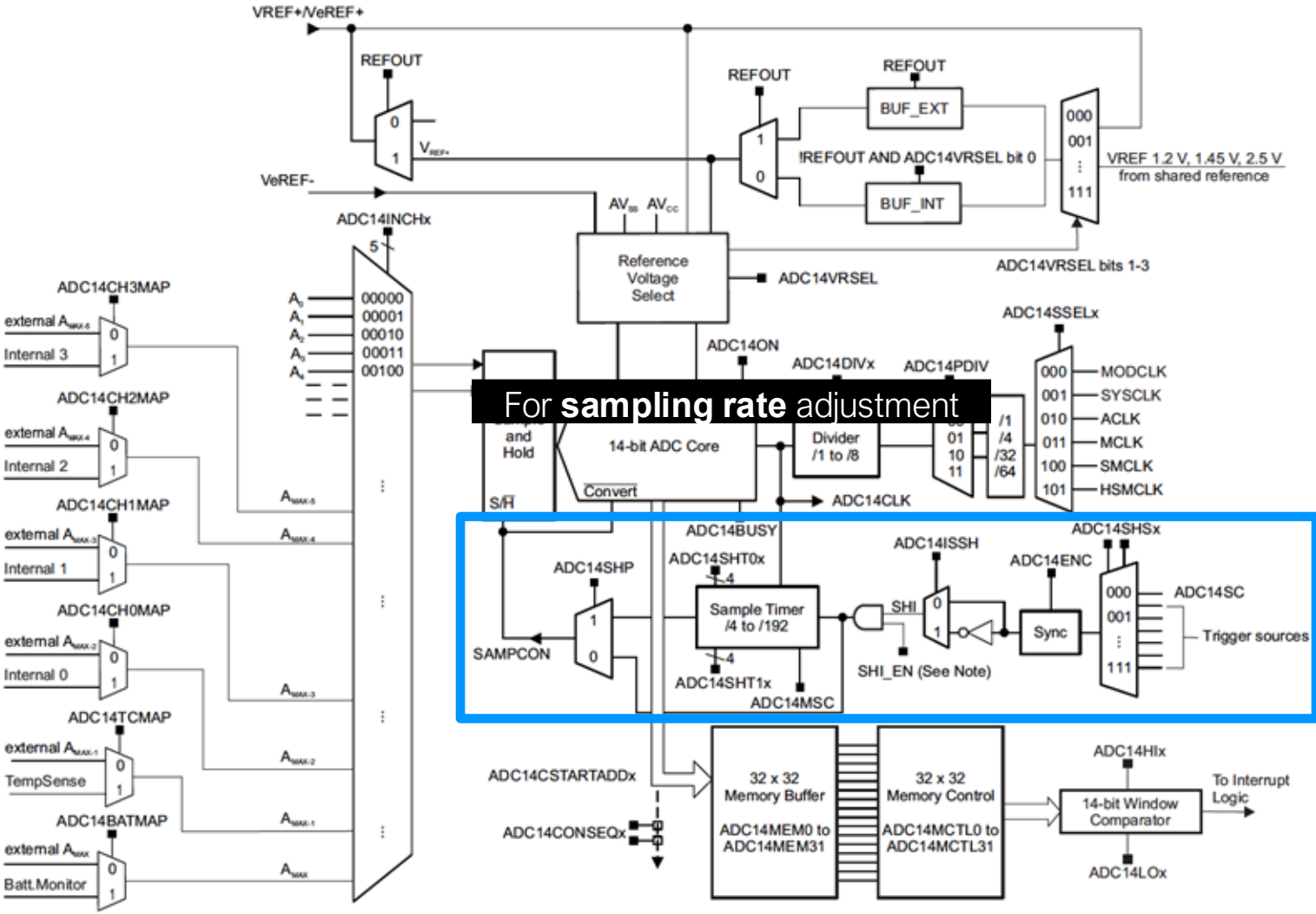
MSP432 ADC


VREF+/VeREF+
REFOUT
REFOUT
REFOUT
BUF_EXT
!REFOUT AND ADC14VRSEL bit 0
BUF_INT
000
001
111
VREF 1.2 V, 1.45 V, 2.5 V
from shared reference
VeREF-
AVSS AVCC
Reference Voltage Select
ADC14VRSEL
ADC14VRSEL bits 1-3
ADC14INCHx
5
ADC14CH3MAP
external A
Internal 3
ADC14CH2MAP
external A
Internal 2
ADC14CH1MAP
external A
Internal 1
ADC14CH0MAP
external A
Internal 0
ADC14TCMAP
external A
TempSense
ADC14BATMAP
external A
Batt.Monitor
00000
00001
00010
00011
00100
32 individually configurable external input channels
ADC14ON
ADC14DIVx
ADC14PDIV
ADC14SSELx
/1
/4
/32
/64
000 MODCLK
001 SYSCLK
010 ACLK
011 MCLK
100 SMCLK
101 HSMCLK
S/H
Convert
ADC14CLK
ADC14BUSY
ADC14SHP
ADC14SHT0x
ADC14ISSH
ADC14ENC
ADC14SHSx
Sample Timer /4 to /192
SHI
SAMPCON
ADC14SHT1x
ADC14MSC
SHI_EN (See Note)
Sync
000
001
111
ADC14SC
Trigger sources
ADC14CSTARTADDx
ADC14CONSEQx
32 x 32 Memory Buffer
ADC14MEM0 to ADC14MEM31
32 x 32 Memory Control
ADC14MCTL0 to ADC14MCTL31
ADC14HIx
14-bit Window Comparator
To Interrupt Logic
ADC14LOx

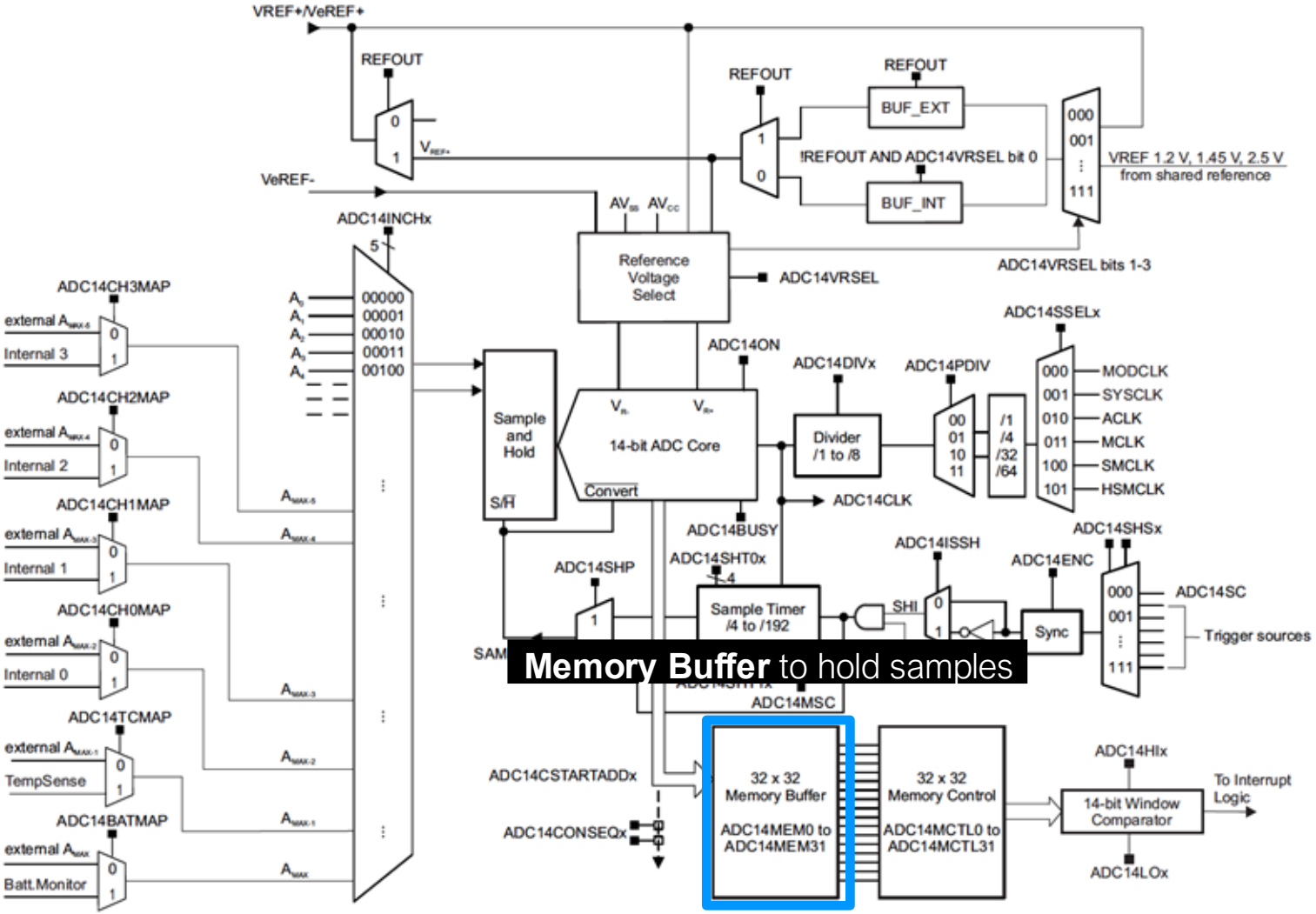
MSP432 ADC


VREF+/VeREF+
REFOUT
VeREF-
ADC14INCHx
5
Reference Voltage Select
ADC14VRSEL
ADC14VRSEL bits 1-3
!REFOUT AND ADC14VRSEL bit 0
BUF_EXT
BUF_INT
VREF 1.2 V, 1.45 V, 2.5 V from shared reference
ADC14CH3MAP
ADC14CH2MAP
ADC14CH1MAP
ADC14CH0MAP
ADC14TCMAP
ADC14BATMAP
external A
Internal 3
Internal 2
Internal 1
Internal 0
TempSense
Batt.Monitor
00000
00001
00010
00011
00100
Internal temperature sensor
ADC14ON
ADC14DIVx
ADC14PDIV
ADC14SSELx
MODCLK
SYSCLK
ACLK
MCLK
SMCLK
HSMCLK
Sample and Hold
S/H
14-bit ADC Core
Convert
Divider /1 to /8
/1
/4
/32
/64
ADC14CLK
ADC14BUSY
ADC14SHP
ADC14SHT0x
ADC14SHT1x
Sample Timer /4 to /192
SAMPCON
ADC14ISSH
SHI
SHI_EN (See Note)
ADC14ENC
Sync
ADC14SHSx
ADC14SC
Trigger sources
ADC14MSC
ADC14CSTARTADDx
ADC14CONSEQx
32 x 32 Memory Buffer
ADC14MEM0 to ADC14MEM31
32 x 32 Memory Control
ADC14MCTL0 to ADC14MCTL31
ADC14HIx
14-bit Window Comparator
To Interrupt Logic
ADC14LOx

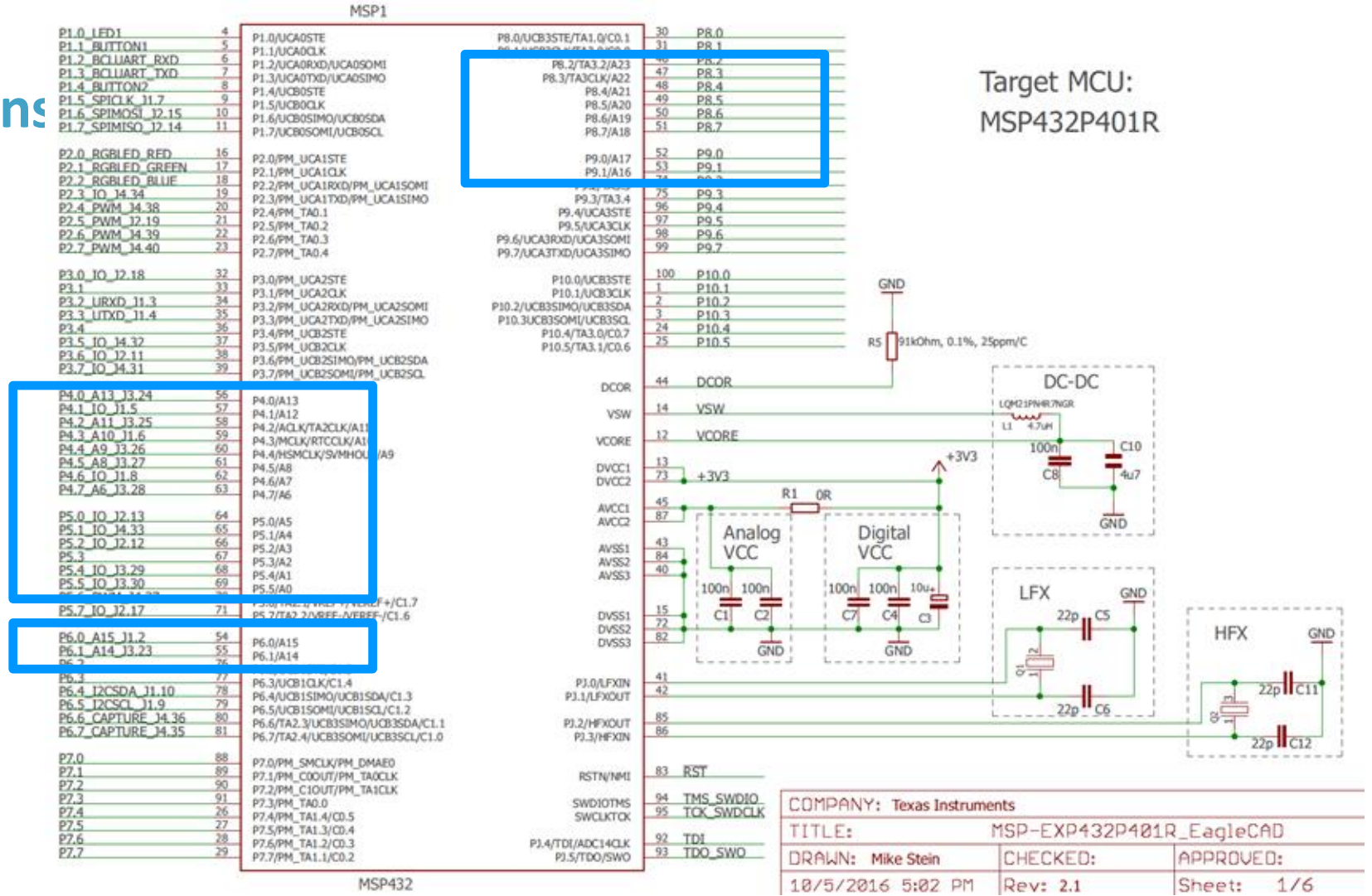
MSP432 ADC


VREF+/VeREF+
REFOUT
VeREF-
ADC14INCHx
5
ADC14CH3MAP
external A
Internal 3
ADC14CH2MAP
external A
Internal 2
ADC14CH1MAP
external A
Internal 1
ADC14CH0MAP
external A
Internal 0
ADC14TCMAP
external A
TempSense
ADC14BATMAP
external A
Batt.Monitor
00000
00001
00010
00011
00100
Reference Voltage Select
ADC14VRSEL
BUF_EXT
BUF_INT
!REFOUT AND ADC14VRSEL bit 0
VREF 1.2 V, 1.45 V, 2.5 V from shared reference
ADC14VRSEL bits 1-3
ADC14ON
ADC14DIVx
ADC14PDIV
ADC14SSELx
MODCLK
SYSCLK
ACLK
MCLK
SMCLK
HSMCLK
For sampling rate adjustment
Sample and Hold
14-bit ADC Core
Divider /1 to /8
Convert
S/H
ADC14CLK
ADC14BUSY
ADC14SHP
ADC14SHT0x
ADC14SHT1x
Sample Timer /4 to /192
SAMPCON
SHI
SHI_EN (See Note)
ADC14ISSH
ADC14ENC
ADC14SHSx
Sync
ADC14SC
Trigger sources
ADC14MSC
ADC14CSTARTADDx
ADC14CONSEQx
32 x 32 Memory Buffer
ADC14MEM0 to ADC14MEM31
32 x 32 Memory Control
ADC14MCTL0 to ADC14MCTL31
ADC14HIx
14-bit Window Comparator
To Interrupt Logic
ADC14LOx

MSP432 ADC


VREF+/VeREF+
REFOUT
REFOUT
REFOUT
BUF_EXT
!REFOUT AND ADC14VRSEL bit 0
BUF_INT
VREF 1.2 V, 1.45 V, 2.5 V from shared reference
VeREF-
ADC14INCHx
Reference Voltage Select
ADC14VRSEL
ADC14VRSEL bits 1-3
ADC14CH3MAP
ADC14CH2MAP
ADC14CH1MAP
ADC14CH0MAP
ADC14TCMAP
ADC14BATMAP
external
Internal 3
Internal 2
Internal 1
Internal 0
TempSense
Batt.Monitor
Sample and Hold
S/H
14-bit ADC Core
Convert
ADC14ON
ADC14DIVx
Divider /1 to /8
ADC14PDIV
ADC14SSELx
MODCLK
SYSCLK
ACLK
MCLK
SMCLK
HSMCLK
ADC14CLK
ADC14BUSY
ADC14SHP
ADC14SHT0x
Sample Timer /4 to /192
ADC14ISSH
ADC14ENC
ADC14SHSx
ADC14SC
Trigger sources
Sync
SHI
SAM
Memory Buffer to hold samples
ADC14MSC
ADC14CSTARTADDx
ADC14CONSEQx
32 x 32 Memory Buffer
ADC14MEM0 to ADC14MEM31
32 x 32 Memory Control
ADC14MCTL0 to ADC14MCTL31
ADC14HIx
14-bit Window Comparator
To Interrupt Logic
ADC14LOx

Pins



Target MCU:
MSP432P401R

COMPANY: Texas Instruments		
TITLE:	MSP-EXP432P401R_EagleCAD	
DRAWN: Mike Stein	CHECKED:	APPROVED:
10/5/2016 5:02 PM	Rev: 2.1	Sheet: 1/6

Sampling Voltage From PIN 5.5

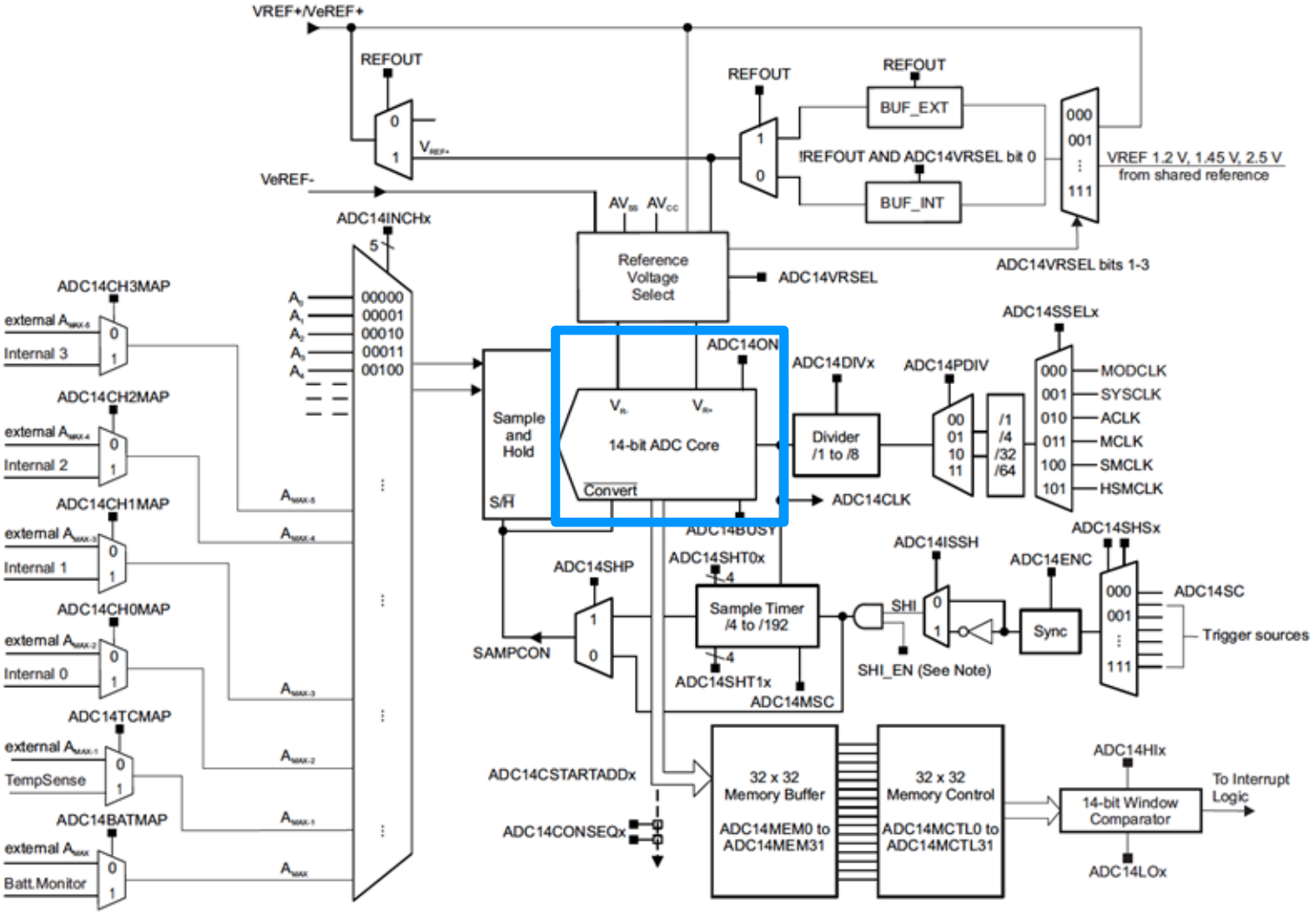
- Configure **Port 5.5** (A0) so that we use it as an input channel to ADC.

```
GPIO_setAsPeripheralModuleFunctionInputPin(GPIO_PORT_P5,GPIO_PIN5,
GPIO_TERTIARY_MODULE_FUNCTION);
```

Sampling Voltage From PIN 5.5

- Enable ADC Hardware.

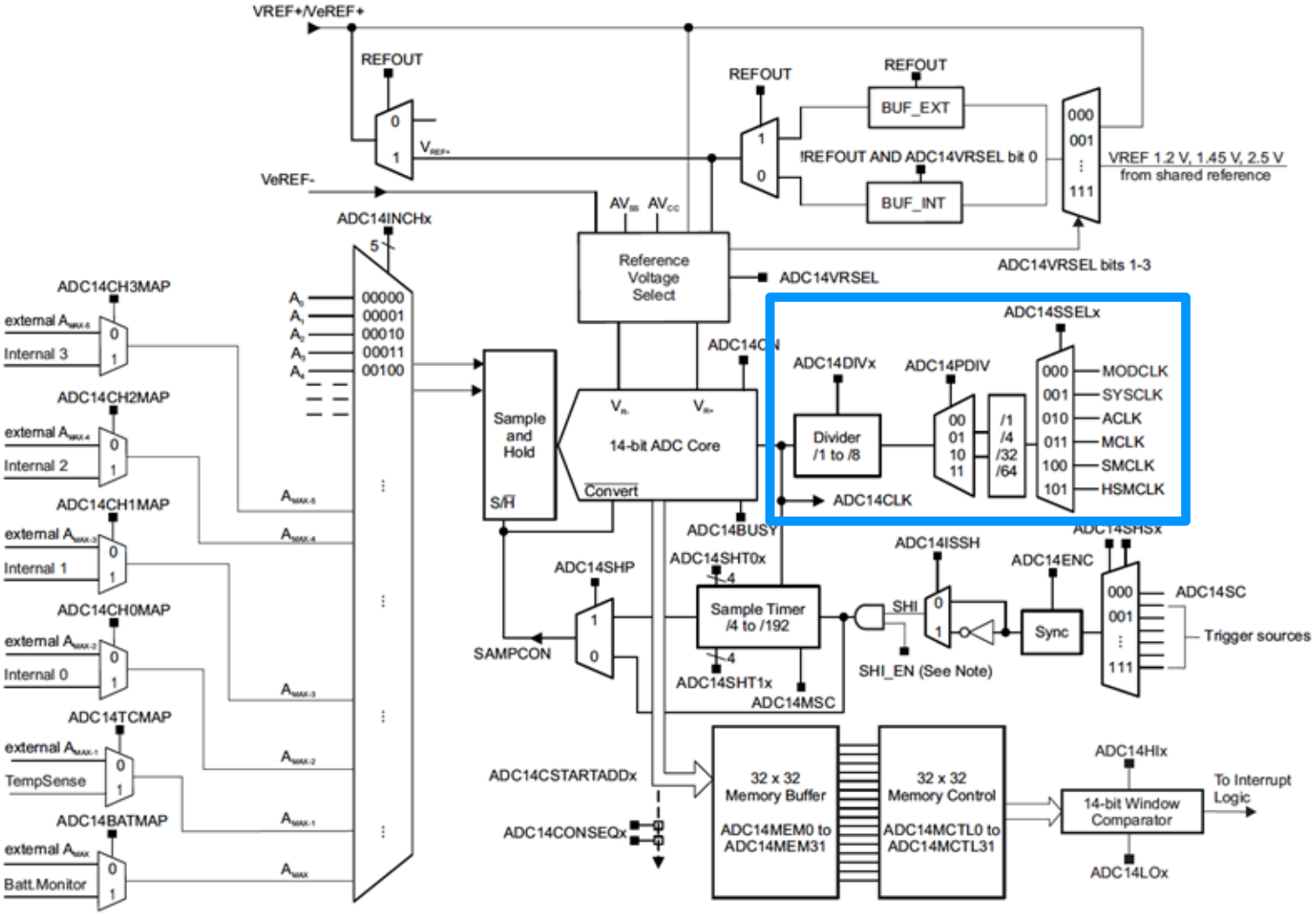
```
GPIO_setAsPeripheralModuleFunctionInputPin(GPIO_PORT_P5,GPIO_PIN5,
GPIO_TERTIARY_MODULE_FUNCTION);
ADC14_enableModule();
```


VREF+/VeREF+
REFOUT
REFOUT
REFOUT
BUF_EXT
!REFOUT AND ADC14VRSEL bit 0
BUF_INT
VREF 1.2 V, 1.45 V, 2.5 V
from shared reference
VeREF-
ADC14INCHx
Reference
Voltage
Select
ADC14VRSEL
ADC14VRSEL bits 1-3
ADC14CH3MAP
ADC14CH2MAP
ADC14CH1MAP
ADC14CH0MAP
ADC14TCMAP
ADC14BATMAP
Internal 3
Internal 2
Internal 1
Internal 0
TempSense
Batt.Monitor
Sample
and
Hold
S/H
14-bit ADC Core
Convert
ADC14ON
ADC14DIVx
Divider
/1 to /8
ADC14PDIV
ADC14SSELx
MODCLK
SYSCLK
ACLK
MCLK
SMCLK
HSMCLK
ADC14CLK
ADC14BUSY
ADC14SHP
ADC14SHT0x
Sample Timer
/4 to /192
ADC14SHT1x
ADC14MSC
SAMPCON
ADC14ISSH
SHI
SHI_EN (See Note)
ADC14ENC
Sync
ADC14SHSx
ADC14SC
Trigger sources
ADC14CSTARTADDx
ADC14CONSEQx
32 x 32
Memory Buffer
ADC14MEM0 to
ADC14MEM31
32 x 32
Memory Control
ADC14MCTL0 to
ADC14MCTL31
ADC14HIx
14-bit Window
Comparator
To Interrupt
Logic
ADC14LOx

Sampling Voltage From PIN 5.5

- Initializes the ADC module and sets up the clock system divider/pre-divider.
  - Set clock source as MCLK/1/4 = 3MHz/4

```
GPIO_setAsPeripheralModuleFunctionInputPin(GPIO_PORT_P5,GPIO_PIN5,
GPIO_TERTIARY_MODULE_FUNCTION);
ADC14_enableModule();
ADC14_initModule(ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1,
ADC_DIVIDER_4,0);
```


VREF+/VeREF+
REFOUT
REFOUT
REFOUT
BUF_EXT
!REFOUT AND ADC14VRSEL bit 0
BUF_INT
000
001
111
VREF 1.2 V, 1.45 V, 2.5 V
from shared reference
VeREF-
ADC14INCHx
5
AVSS AVCC
Reference
Voltage
Select
ADC14VRSEL
ADC14VRSEL bits 1-3
ADC14SSELx
ADC14CH3MAP
external A
Internal 3
ADC14CH2MAP
external A
Internal 2
ADC14CH1MAP
external A
Internal 1
ADC14CH0MAP
external A
Internal 0
ADC14TCMAP
external A
TempSense
ADC14BATMAP
external A
Batt.Monitor
00000
00001
00010
00011
00100
Sample
and
Hold
S/H
14-bit ADC Core
Convert
ADC14DIVx
Divider
/1 to /8
ADC14PDIV
00
01
10
11
/1
/4
/32
/64
000 MODCLK
001 SYSCLK
010 ACLK
011 MCLK
100 SMCLK
101 HSMCLK
ADC14CLK
ADC14BUSY
ADC14SHP
ADC14SHT0x
4
Sample Timer
/4 to /192
4
ADC14SHT1x
ADC14MSC
SAMPCON
ADC14ISSH
SHI
SHI_EN (See Note)
ADC14ENC
Sync
ADC14SHSx
000
001
111
ADC14SC
Trigger sources
ADC14CSTARTADDx
ADC14CONSEQx
32 x 32
Memory Buffer
ADC14MEM0 to
ADC14MEM31
32 x 32
Memory Control
ADC14MCTL0 to
ADC14MCTL31
ADC14HIx
14-bit Window
Comparator
To Interrupt
Logic
ADC14LOx

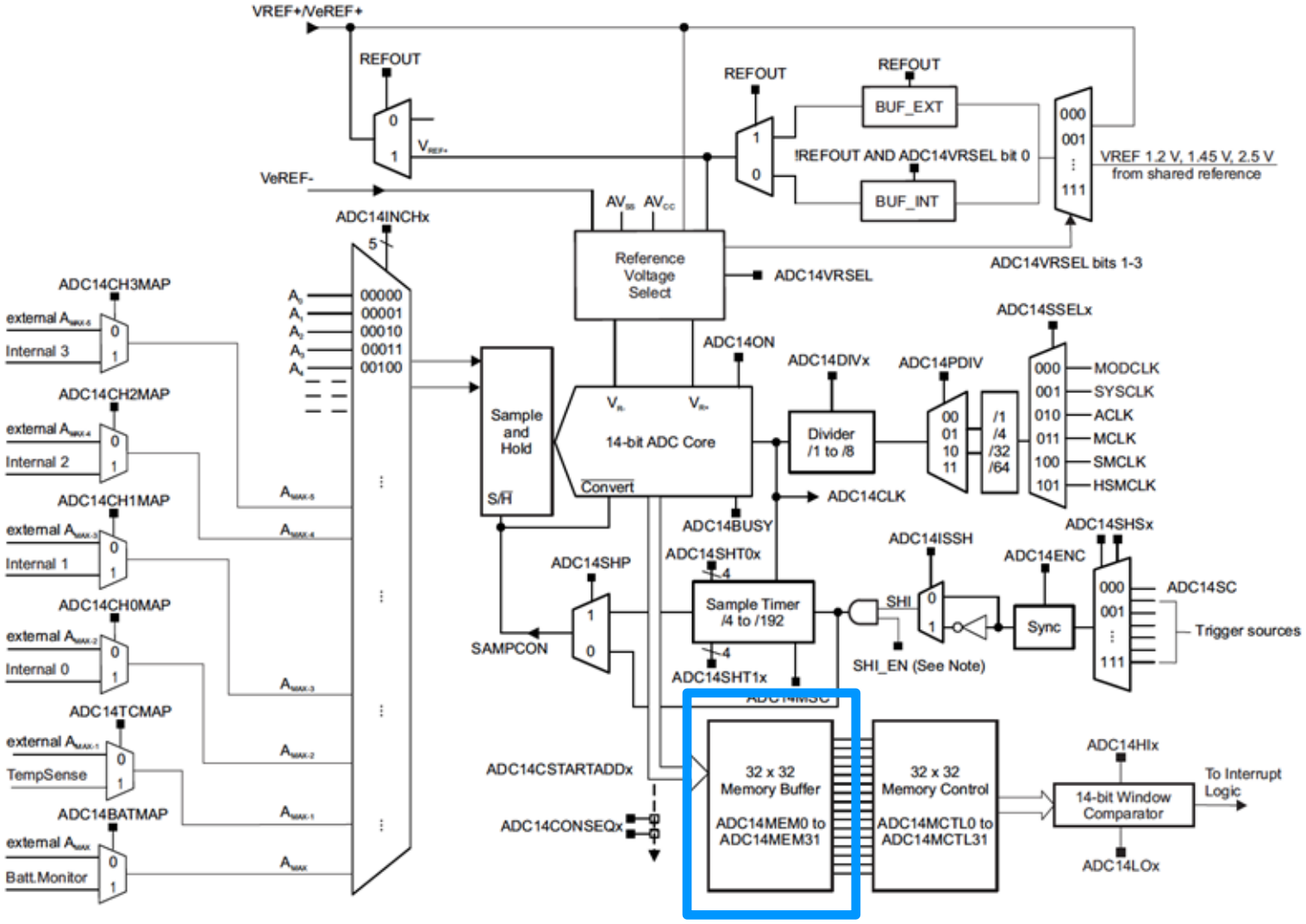
Sampling Voltage From PIN 5.5

- Use a single ADC memory location for sampling/conversion.

```
GPIO_setAsPeripheralModuleFunctionInputPin(GPIO_PORT_P5,GPIO_PIN5,
GPIO_TERTIARY_MODULE_FUNCTION);
ADC14_enableModule();
ADC14_initModule(ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1,
ADC_DIVIDER_4,0);
ADC14_configureSingleSampleMode(ADC_MEM0, true);
```



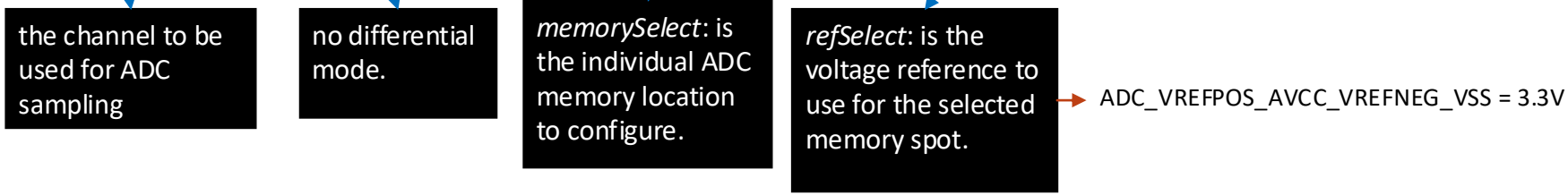
Specifies whether or not to **repeat** the conversion/sample cycle after the first round of sample/conversions

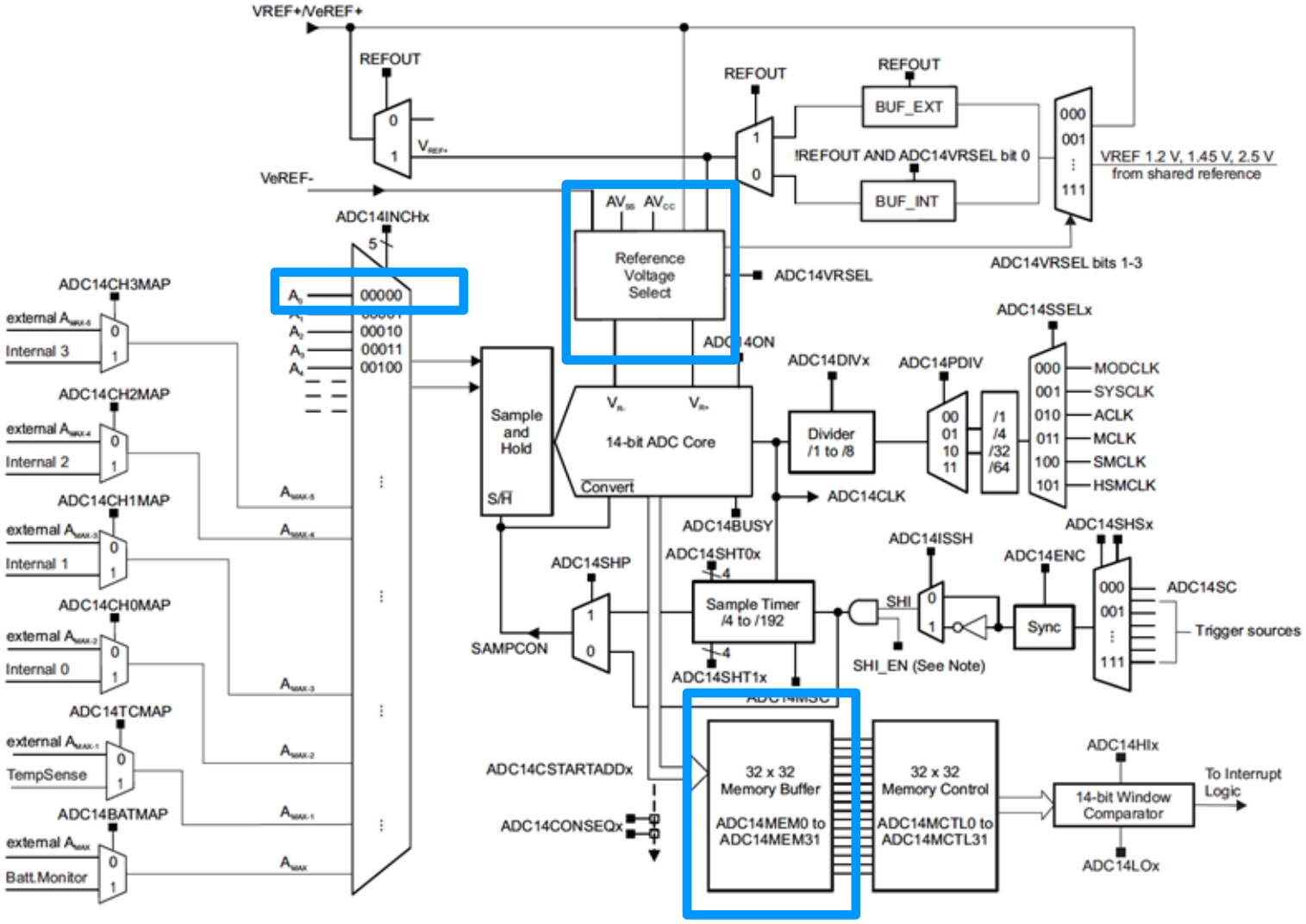

VREF+/VeREF+
REFOUT
VeREF-
REFOUT
REFOUT
BUF_EXT
!REFOUT AND ADC14VRSEL bit 0
BUF_INT
VREF 1.2 V, 1.45 V, 2.5 V
from shared reference
ADC14VRSEL bits 1-3
ADC14INCHx
Reference
Voltage
Select
ADC14VRSEL
ADC14SSELx
ADC14ON
ADC14DIVx
ADC14PDIV
MODCLK
SYSCLK
ACLK
MCLK
SMCLK
HSMCLK
Sample
and
Hold
S/H
14-bit ADC Core
Convert
Divider
/1 to /8
ADC14CLK
ADC14BUSY
ADC14SHT0x
ADC14SHT1x
ADC14SHP
SAMPCON
Sample Timer
/4 to /192
ADC14ISSH
SHI
SHI_EN (See Note)
ADC14ENC
Sync
ADC14SHSx
ADC14SC
Trigger sources
ADC14CH3MAP
external
Internal 3
ADC14CH2MAP
Internal 2
ADC14CH1MAP
Internal 1
ADC14CH0MAP
Internal 0
ADC14TCMAP
TempSense
ADC14BATMAP
Batt.Monitor
ADC14CSTARTADDx
ADC14CONSEQx
32 x 32
Memory Buffer
ADC14MEM0 to
ADC14MEM31
32 x 32
Memory Control
ADC14MCTL0 to
ADC14MCTL31
ADC14HIx
14-bit Window
Comparator
To Interrupt
Logic
ADC14LOx

Sampling Voltage From PIN 5.5

- Configures an individual memory location for the ADC module.

```
GPIO_setAsPeripheralModuleFunctionInputPin(GPIO_PORT_P5,GPIO_PIN5,
GPIO_TERTIARY_MODULE_FUNCTION);
ADC14_enableModule();
ADC14_initModule(ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1,
ADC_DIVIDER_4,0);
ADC14_configureSingleSampleMode(ADC_MEM0, true);
ADC14_configureConversionMemory(ADC_MEM0, ADC_VREFPOS_AVCC_VREFNEG_VSS,
ADC_INPUT_A0, false);
```




VREF+/VeREF+
REFOUT
VeREF-
ADC14INCHx
5
ADC14CH3MAP
external A
Internal 3
ADC14CH2MAP
Internal 2
ADC14CH1MAP
Internal 1
ADC14CH0MAP
Internal 0
ADC14TCMAP
TempSense
ADC14BATMAP
Batt.Monitor
00000
00010
00011
00100
Reference Voltage Select
AVSS AVCC
ADC14VRSEL
ADC14ON
BUF_EXT
BUF_INT
!REFOUT AND ADC14VRSEL bit 0
VREF 1.2 V, 1.45 V, 2.5 V from shared reference
ADC14VRSEL bits 1-3
Sample and Hold
S/H
14-bit ADC Core
Convert
ADC14DIVx
Divider /1 to /8
ADC14PDIV
/1 /4 /32 /64
ADC14SSELx
MODCLK
SYSCLK
ACLK
MCLK
SMCLK
HSMCLK
ADC14CLK
ADC14BUSY
ADC14SHP
SAMPCON
ADC14SHT0x
ADC14SHT1x
Sample Timer /4 to /192
ADC14ISSH
SHI
SHI_EN (See Note)
ADC14ENC
Sync
ADC14SHSx
ADC14SC
Trigger sources
ADC14CSTARTADDx
ADC14CONSEQx
32 x 32 Memory Buffer
ADC14MEM0 to ADC14MEM31
32 x 32 Memory Control
ADC14MCTL0 to ADC14MCTL31
ADC14HIx
14-bit Window Comparator
To Interrupt Logic
ADC14LOx

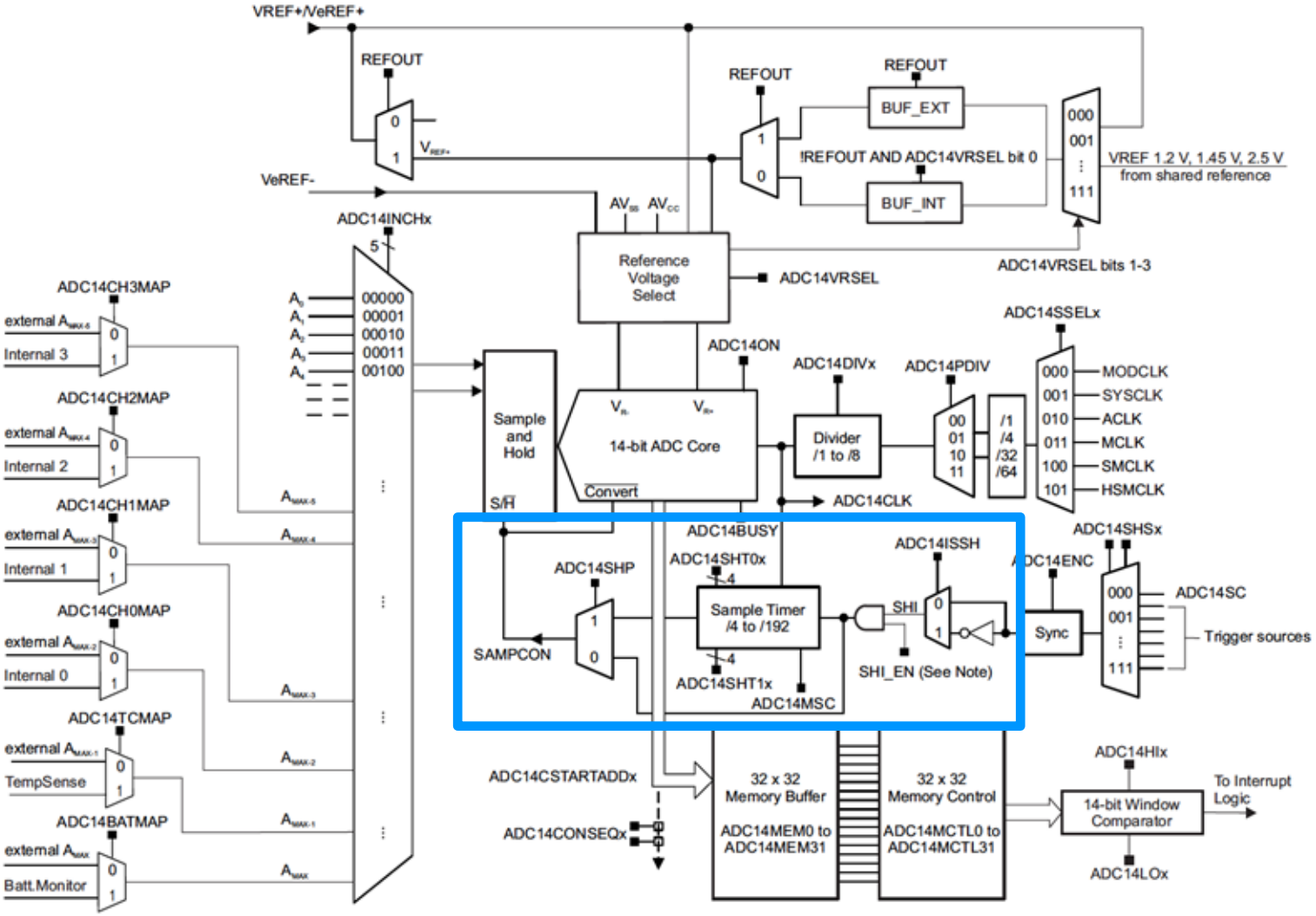
Sampling Voltage From PIN 5.5

- Enables SAMPCON to be sourced from the sampling timer

```
GPIO_setAsPeripheralModuleFunctionInputPin(GPIO_PORT_P5,GPIO_PIN5,
GPIO_TERTIARY_MODULE_FUNCTION);
ADC14_enableModule();
ADC14_initModule(ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1,
ADC_DIVIDER_4,0);
ADC14_configureSingleSampleMode(ADC_MEM0, true);
ADC14_configureConversionMemory(ADC_MEM0, ADC_VREFPOS_AVCC_VREFNEG_VSS,
ADC_INPUT_A0, false);
ADC14_enableSampleTimer(ADC_MANUAL_ITERATION);
```

The user will have to manually set the SHI signal (usually by *ADC14_toggleConversionTrigger*) at the end of each sample/conversion cycle.

SHI: Sample and Hold Indicator signal


VREF+/VeREF+
REFOUT
VeREF-
ADC14INCHx
5
ADC14CH3MAP
external A
Internal 3
ADC14CH2MAP
Internal 2
ADC14CH1MAP
Internal 1
ADC14CH0MAP
Internal 0
ADC14TCMAP
TempSense
ADC14BATMAP
Batt.Monitor
00000
00001
00010
00011
00100
Reference Voltage Select
ADC14VRSEL
AVSS AVCC
REFOUT
BUF_EXT
!REFOUT AND ADC14VRSEL bit 0
BUF_INT
000
001
111
VREF 1.2 V, 1.45 V, 2.5 V from shared reference
ADC14VRSEL bits 1-3
ADC14SSELx
000 MODCLK
001 SYSCLK
010 ACLK
011 MCLK
100 SMCLK
101 HSMCLK
ADC14ON
ADC14DIVx
ADC14PDIV
00
01
10
11
/1
/4
/32
/64
Sample and Hold
14-bit ADC Core
Convert
Divider /1 to /8
ADC14CLK
S/H
ADC14BUSY
ADC14SHT0x
ADC14SHP
Sample Timer /4 to /192
ADC14ISSH
SHI
SHI_EN (See Note)
SAMPCON
ADC14SHT1x
ADC14MSC
ADC14SHSx
ADC14ENC
Sync
ADC14SC
Trigger sources
ADC14CSTARTADDx
ADC14CONSEQx
32 x 32 Memory Buffer
ADC14MEM0 to ADC14MEM31
32 x 32 Memory Control
ADC14MCTL0 to ADC14MCTL31
ADC14HIx
14-bit Window Comparator
To Interrupt Logic
ADC14LOx

Sampling Voltage From PIN 5.5

- Enable conversion of ADC data, ADC to start conversion and ADC interrupts.

```
GPIO_setAsPeripheralModuleFunctionInputPin(GPIO_PORT_P5,GPIO_PIN5,
GPIO_TERTIARY_MODULE_FUNCTION);
ADC14_enableModule();
ADC14_initModule(ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1,
ADC_DIVIDER_4,0);
ADC14_configureSingleSampleMode(ADC_MEM0, true);
ADC14_configureConversionMemory(ADC_MEM0, ADC_VREFPOS_AVCC_VREFNEG_VSS,
ADC_INPUT_A0, false);
ADC14_enableSampleTimer(ADC_MANUAL_ITERATION);
ADC14_enableConversion();
ADC14_toggleConversionTrigger();
ADC14_enableInterrupt(ADC_INT0);
Interrupt_enableInterrupt(INT_ADC14);
Interrupt_enableMaster();
```

Sampling Voltage From PIN 5.5

- ADC Interrupt Service Routine

```
void ADC14_IRQHandler(void){
    uint64_t status = ADC14_getEnabledInterruptStatus();
    ADC14_clearInterruptFlag(status);

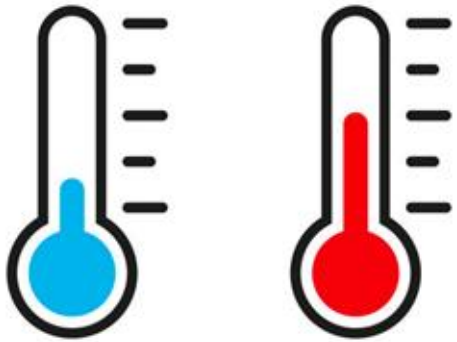
    if (ADC_INT0 & status){
        /* should be between 0 and 16384*/
        curADCResult = ADC14_getResult(ADC_MEM0);

        if(curADCResult > 16000)
            GPIO_setOutputHighOnPin(GPIO_PORT_P1,GPIO_PIN0);
        else
            GPIO_setOutputLowOnPin(GPIO_PORT_P1,GPIO_PIN0);

        /* next sample */
        ADC14_toggleConversionTrigger();
    }
}
```

Temperature Sensing

- Periodically sample values from the temperature sensor.
  - We will read voltage values using analog-to-digital converter.



Voltage Reference Module

- Generates reference voltages for use by various analog modules in a device.
  - 1.2 V, 1.45 V, and 2.5 V.
- Includes a **temperature sensor**
  - Can used by an ADC to measure a voltage proportional to temperature.

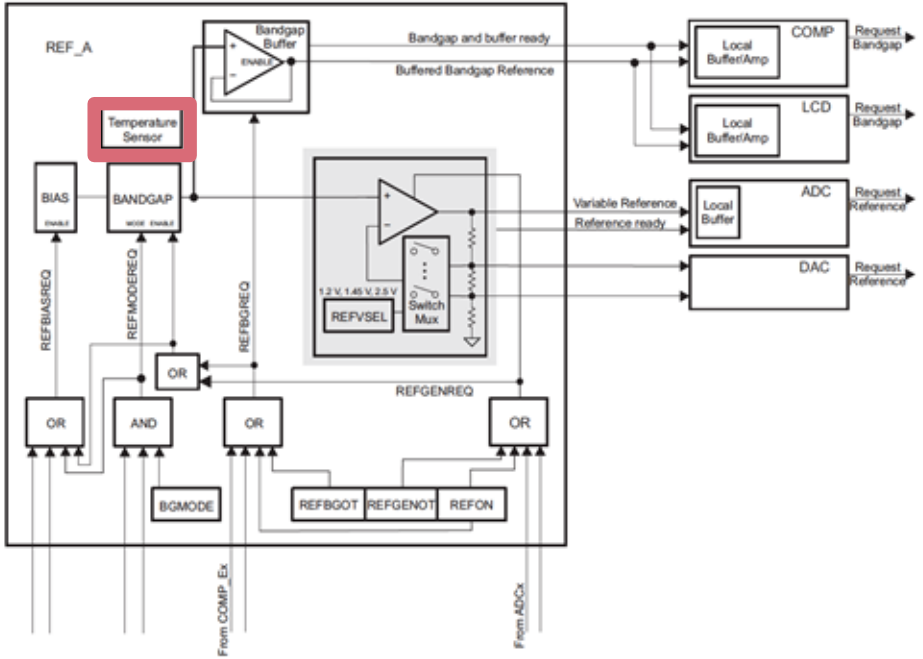


Figure 21-1. REF_A Block Diagram

On-chip Temperature Sensor

- Temperature sensor circuitry:
  - voltage proportional to temperature.

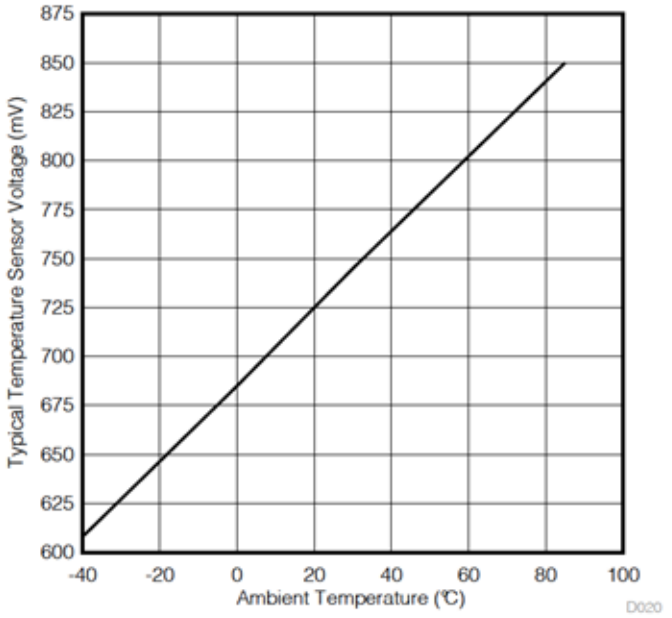


Figure 22-11. Typical Temperature Sensor Transfer Function

On-chip Temperature Sensor

- The temperature sensor is calibrated using the **internal voltage references**.
  - Each reference voltage (1.2V,1.45V, or 2.5V) contains a measured value for two temperatures:
    - 30°C ±3°C and 85°C ±3°C
- The temperature (Temp, °C) can be computed as follows for each of the reference voltages used in the ADC measurement:

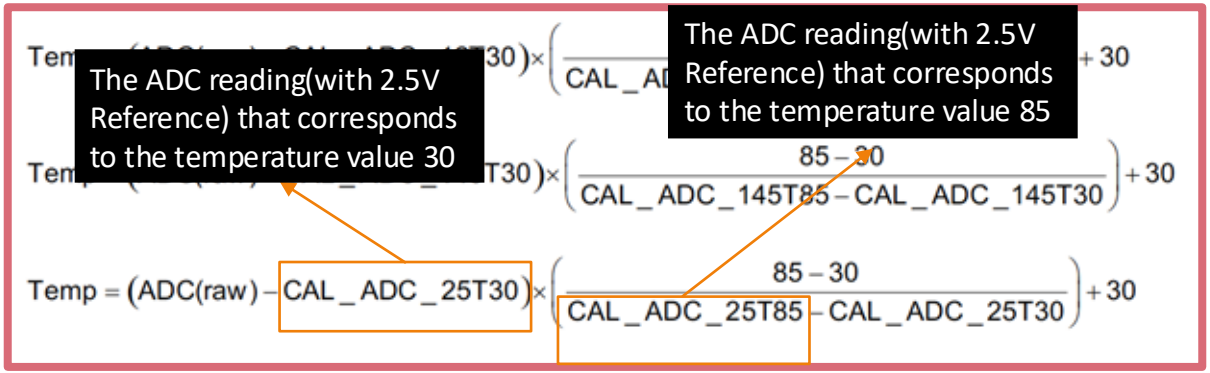
$$\text{Temp} = (\text{ADC(raw)} - \text{CAL_ADC_12T30}) \times \left(\frac{85 - 30}{\text{CAL_ADC_12T85} - \text{CAL_ADC_12T30}} \right) + 30$$

$$\text{Temp} = (\text{ADC(raw)} - \text{CAL_ADC_145T30}) \times \left(\frac{85 - 30}{\text{CAL_ADC_145T85} - \text{CAL_ADC_145T30}} \right) + 30$$

$$\text{Temp} = (\text{ADC(raw)} - \text{CAL_ADC_25T30}) \times \left(\frac{85 - 30}{\text{CAL_ADC_25T85} - \text{CAL_ADC_25T30}} \right) + 30$$

On-chip Temperature Sensor

- The temperature sensor is calibrated using the **internal voltage references**.
  - Each reference voltage (1.2V,1.45V, or 2.5V) contains a measured value for two temperatures:
    - 30°C ±3°C and 85°C ±3°C
- The temperature (Temp, °C) can be computed as follows for each of the reference voltages used in the ADC measurement:



On-chip Temperature Sensor

```
/* Setting reference voltage to 2.5 and
enabling temperature sensor */
REF_A_enableTempSensor();
REF_A_setReferenceVoltage(REF_A_VREF2_5V);
REF_A_enableReferenceVoltage();
```

On-chip Temperature Sensor

```
/* Setting reference voltage to 2.5 and
enabling temperature sensor */
REF_A_enableTempSensor();
REF_A_setReferenceVoltage(REF_A_VREF2_5V);
REF_A_enableReferenceVoltage()
```

void REF_A_enableTempSensor (void)

Enables the internal temperature sensor.

This function is used to turn on the internal temperature sensor to use by other peripherals. The temperature sensor is enabled by default. Please note, if the REF_A_isRefGenBusy() returns **REF_A_BUSY**, this function will have no effect.

Modified bits are **REFTCOFF** of **REFCTL0** register.

Returns

none

On-chip Temperature Sensor

```
/* Setting reference voltage to 2.5 and
enabling temperature sensor */
REF_A_enableTempSensor();
REF_A_setReferenceVoltage(REF_A_VREF2_5V);
REF_A_enableReferenceVoltage();
```

void REF_A_setReferenceVoltage (uint_fast8_t *referenceVoltageSelect*)

Sets the reference voltage for the voltage generator.

Parameters

referenceVoltageSelect	is the desired voltage to generate for a reference voltage. Valid values are: ■ **REF_A_VREF1_2V** [Default] ■ **REF_A_VREF1_45V** ■ **REF_A_VREF2_5V** Modified bits are **REFVSEL** of **REFCTL0** register.

This function sets the reference voltage generated by the voltage generator to be used by other peripherals. This reference voltage will only be valid while the REF module is in control. Please note, if the REF_A_isRefGenBusy() returns **REF_BUSY**, this function will have no effect.

Returns
none

On-chip Temperature Sensor

```
/* Setting reference voltage to 2.5 and
enabling temperature sensor */
REF_A_enableTempSensor();
REF_A_setReferenceVoltage(REF_A_VREF2_5V);
REF_A_enableReferenceVoltage();
```

void REF_A_enableReferenceVoltage (void)

Enables the reference voltage to be used by peripherals.

This function is used to enable the generated reference voltage to be used other peripherals or by an output pin, if enabled. Please note, that giving ADC12 module control over the REF module, the state of the reference voltage is dependent on the controls of the ADC12 module. Please note, if the REF_A_isRefGenBusy() returns REF_A_BUSY, this function will have no effect.

Modified bits are **REFON** of **REFCTL0** register.

On-chip Temperature Sensor

- We need to know which voltage values are used for 30 and 85 degrees

```
uint32_t cal30;
uint32_t cal85;
float calDifference;
...

cal30 = SysCtl_getTempCalibrationConstant(SYSCTL_2_5V_REF,SYSCTL_30_DEGREES_C);
cal85 = SysCtl_getTempCalibrationConstant(SYSCTL_2_5V_REF,SYSCTL_85_DEGREES_C);
calDifference = cal85 - cal30;
```

On-chip Temperature Senso

- We need to know which voltage v
  degrees

uint_fast16_t SysCtl_getTempCalibrationConstant (uint32_t *refVoltage*, uint32_t *temperature*)

Retrieves the calibration constant of the temperature sensor to be used in temperature calculation.

Parameters

refVoltage	Reference voltage being used.

The *refVoltage* parameter must be only one of the following values:

- **SYSCTL_1_2V_REF**
- **SYSCTL_1_45V_REF**
- **SYSCTL_2_5V_REF**

Parameters

temperature	is the calibration temperature that the user wants to be returned.

The *temperature* parameter must be only one of the following values:

- **SYSCTL_30_DEGREES_C**
- **SYSCTL_85_DEGREES_C**

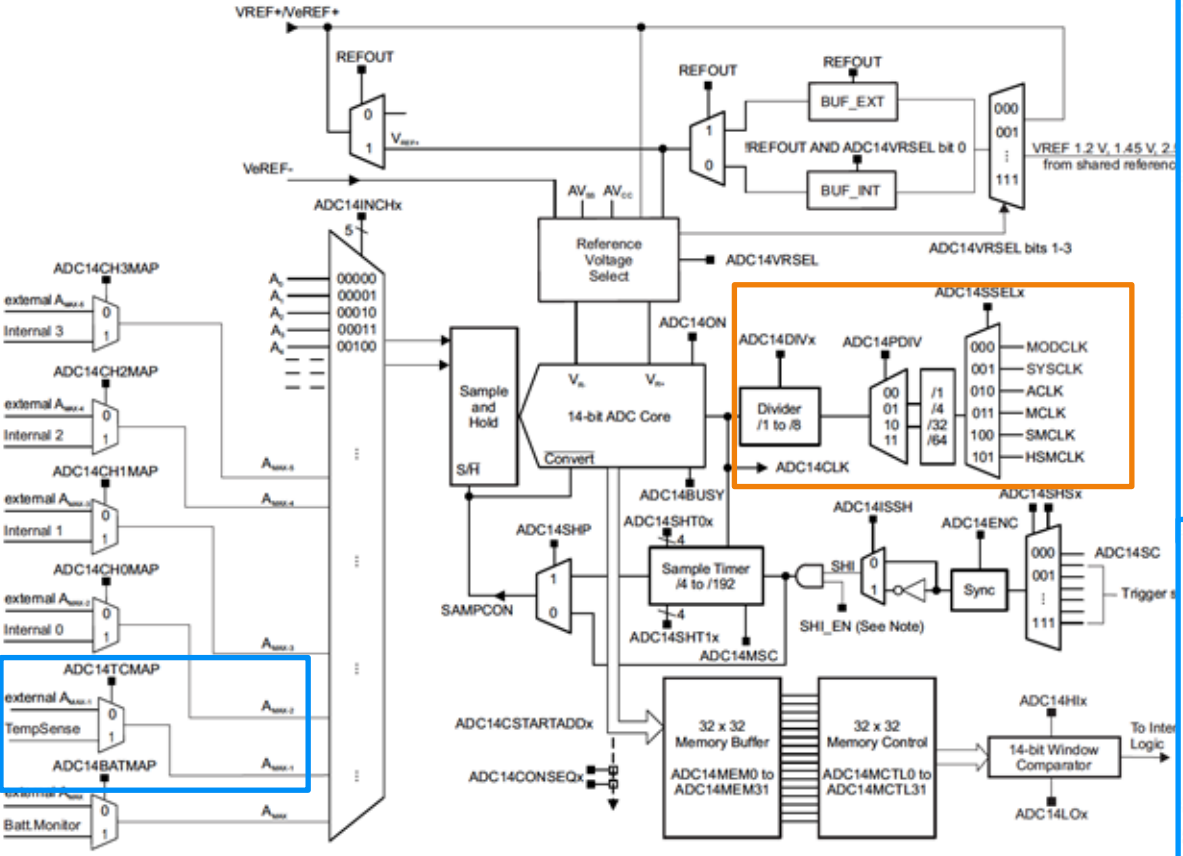
Returns
None.

```
uint32_t cal30;
uint32_t cal85;
float calDifference;
...

cal30 = SysCtl_getTempCalibrationConstant(SYSCTL_2_5V_REF,SYSCTL_30_DEGREES_C);
cal85 = SysCtl_getTempCalibrationConstant(SYSCTL_2_5V_REF,SYSCTL_85_DEGREES_C);
calDifference = cal85 - cal30;
```

On-chip Temperature Sensor

```
/* Initializing ADC (MCLK/1/1) with temperature sensor routed */
ADC14_enableModule();
ADC14_initModule(ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1,
ADC_DIVIDER_1,ADC_TEMPSENSEMAP);
```



bool ADC14_initModule (uint32_t *clockSource*, uint32_t *clockPredivider*, uint32_t *clockDivider*, uint32_t *internalChannelMask*)

Initializes the ADC module and sets up the clock system divider/pre-divider. This initialization function will also configure the internal/external signal mapping.

Note

A call to this function while active ADC conversion is happening is an invalid case and will result in a false value being returned.

Parameters

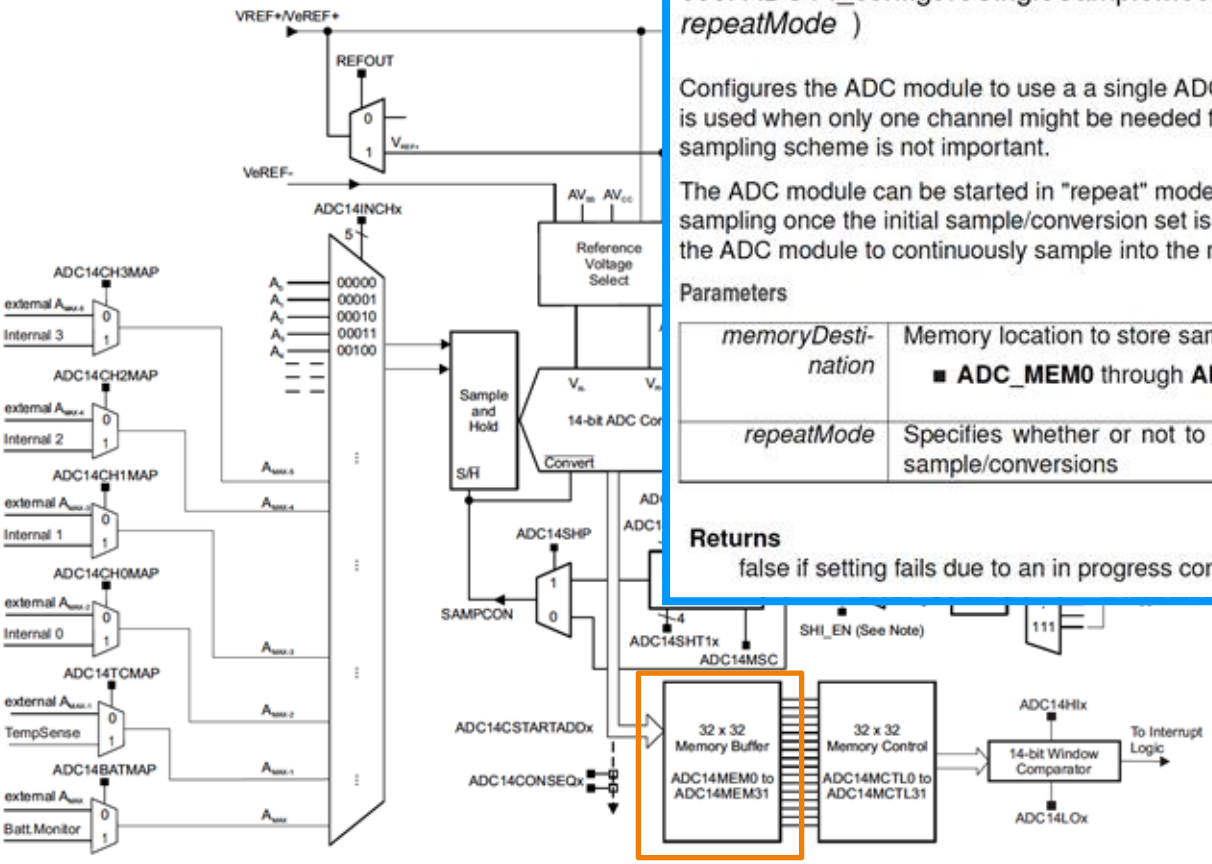
clockSource	The clock source to use for the ADC module. ■ **ADC_CLOCKSOURCE_ADCOSC** [DEFAULT] ■ **ADC_CLOCKSOURCE_SYSOSC** ■ **ADC_CLOCKSOURCE_ACLK** ■ **ADC_CLOCKSOURCE_MCLK** ■ **ADC_CLOCKSOURCE_SMCLK** ■ **ADC_CLOCKSOURCE_HSMCLK**
clockPredivider	Divides the given clock source before feeding it into the main clock divider. Valid values are: ■ **ADC_PREDIVIDER_1** [DEFAULT] ■ **ADC_PREDIVIDER_4** ■ **ADC_PREDIVIDER_32** ■ **ADC_PREDIVIDER_64**
clockDivider	Divides the pre-divided clock source Valid values are ■ **ADC_DIVIDER_1** [Default value] ■ **ADC_DIVIDER_2** ■ **ADC_DIVIDER_3** ■ **ADC_DIVIDER_4** ■ **ADC_DIVIDER_5** ■ **ADC_DIVIDER_6** ■ **ADC_DIVIDER_7** ■ **ADC_DIVIDER_8**
internalChannelMask	Configures the internal/external pin mappings for the ADC modules. This setting determines if the given ADC channel or component is mapped to an external pin (default), or routed to an internal component. This parameter is a bit mask where a logical high value will switch the component to the internal routing. For a list of internal routings, please refer to the device specific data sheet. Valid values are a logical OR of the following values: ■ **ADC_MAPINTCH3** ■ **ADC_MAPINTCH2** ■ **ADC_MAPINTCH1** ■ **ADC_MAPINTCH0** ■ **ADC_TEMPSENSEMAP** ■ **ADC_BATTMAP** ■ ADC_NOROUTE If internalChannelMask is not desired, pass ADC_NOROUTE in lieu of this parameter.

Returns

false if the initialization fails due to an in progress conversion

On-chip Temperature Sensor

```
/* Initializing ADC (MCLK/1/1) with temperature sensor routed */
ADC14_enableModule();
ADC14_initModule(ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1,
ADC_DIVIDER_1,ADC_TEMPSENSEMAP);
/* Configuring ADC Memory (ADC_MEM0 A22 (Temperature Sensor) in repeat mode */
ADC14_configureSingleSampleMode(ADC_MEM0, true);
ADC14_configureConversionMemory(ADC_MEM0,
ADC_VREFPOS_INTBUF_VREFNEG_VSS,ADC_INPUT_A22, false);
```



bool ADC14_configureSingleSampleMode (uint32_t *memoryDestination*, bool *repeatMode*)

Configures the ADC module to use a a single ADC memory location for sampling/conversion. This is used when only one channel might be needed for conversion, or where using a multiple sampling scheme is not important.

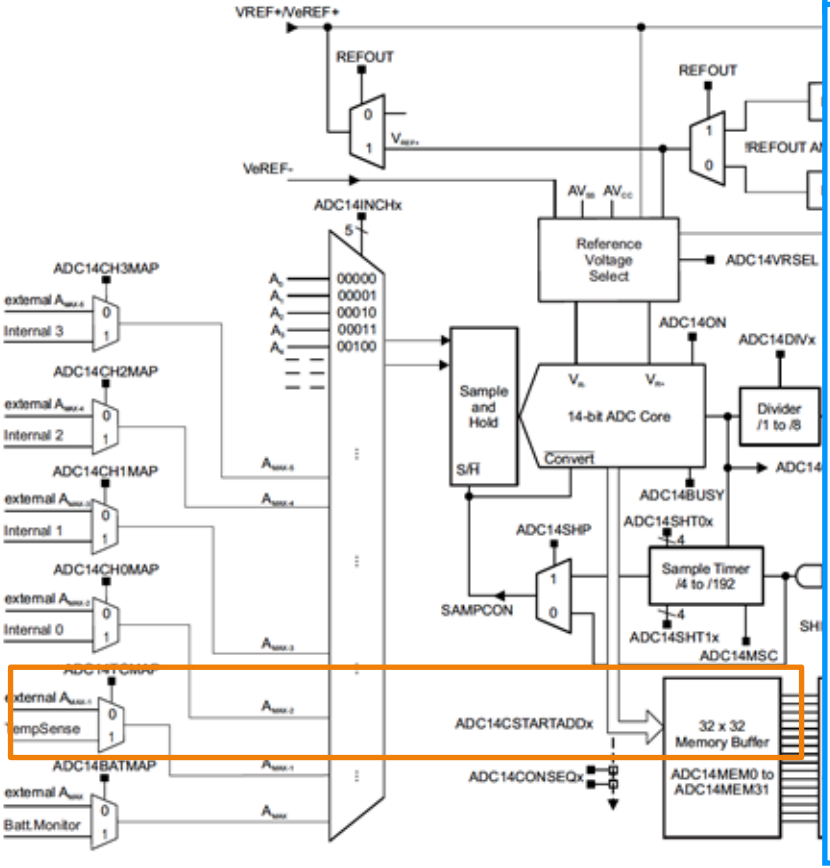
The ADC module can be started in "repeat" mode which will cause the ADC module to resume sampling once the initial sample/conversion set is executed. In single sample mode, this will cause the ADC module to continuously sample into the memory destination provided.

Parameters

memoryDestination	Memory location to store sample/conversion value. Possible values include: ■ **ADC_MEM0** through **ADC_MEM31**
repeatMode	Specifies whether or not to repeat the conversion/sample cycle after the first round of sample/conversions

Returns

false if setting fails due to an in progress conversion



bool ADC14_configureConversionMemory (uint32_t *memorySelect*, uint32_t *refSelect*, uint32_t *channelSelect*, bool *differntialMode*)

Configures an individual memory location for the ADC module.

Parameters

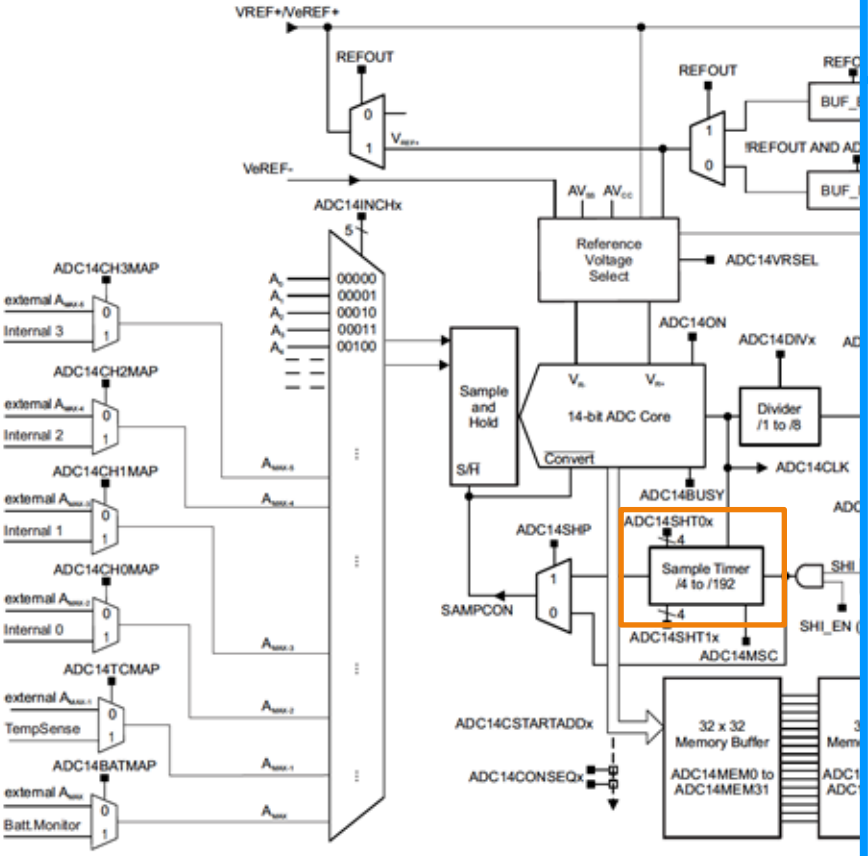
memorySelect	is the individual ADC memory location to configure. If multiple memory locations want to be configured with the same configuration, this value can be logically ORed together with other values. ■ **ADC_MEM0** through **ADC_MEM31**
refSelect	is the voltage reference to use for the selected memory spot. Possible values include: ■ **ADC_VREFPOS_AVCC_VREFNEG_VSS** [DEFAULT] ■ **ADC_VREFPOS_INTBUF_VREFNEG_VSS** ■ **ADC_VREFPOS_EXTPOS_VREFNEG_EXTNEG** ■ **ADC_VREFPOS_EXTBUF_VREFNEG_EXTNEG**
channelSelect	selects the channel to be used for ADC sampling. Note if differential mode is enabled, the value sampled will be equal to the difference between the corresponding even/odd memory locations. Possible values are: ■ **ADC_INPUT_A0** through **ADC_INPUT_A31**
differntialMode	selects if the channel selected by the channelSelect will be configured in differential mode. If this parameter is given as true, the configured channel will be paired with its neighbor in differential mode. for example, if channel A0 or A1 is selected, the channel configured will be the difference between A0 and A1. If A2 or A3 are selected, the channel configured will be the difference between A2 and A3 (and so on). Users can enter true or false, or one of the following values: ■ ADC_NONDIFFERENTIAL_INPUTS ■ ADC_DIFFERENTIAL_INPUTS

Returns

false if setting fails due to an in progress conversion

On-chip Temperature Sensor

```
/* Initializing ADC (MCLK/1/1) with temperature sensor routed */
ADC14_enableModule();
ADC14_initModule(ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1,
ADC_DIVIDER_1,ADC_TEMPSENSEMAP);
/* Configuring ADC Memory (ADC_MEM0 A22 (Temperature Sensor) in repeat mode */
ADC14_configureSingleSampleMode(ADC_MEM0, true);
ADC14_configureConversionMemory(ADC_MEM0,
ADC_VREFPOS_INTBUF_VREFNEG_VSS,ADC_INPUT_A22, false);
/* Configuring the sample/hold time for 192 */
ADC14_setSampleHoldTime(ADC_PULSE_WIDTH_192,ADC_PULSE_WIDTH_192);
```



bool ADC14_setSampleHoldTime (uint32_t *firstPulseWidth*, uint32_t *secondPulseWidth*)

Sets the sample/hold time for the specified memory register range. The duration of time required for a sample differs depending on the user's hardware configuration.

There are two values in the ADCC module. The first value controls ADC memory locations ADC_MEMORY_0 through ADC_MEMORY_7 and ADC_MEMORY_24 through ADC_MEMORY_31, while the second value controls memory locations ADC_MEMORY_8 through ADC_MEMORY_23.

Parameters

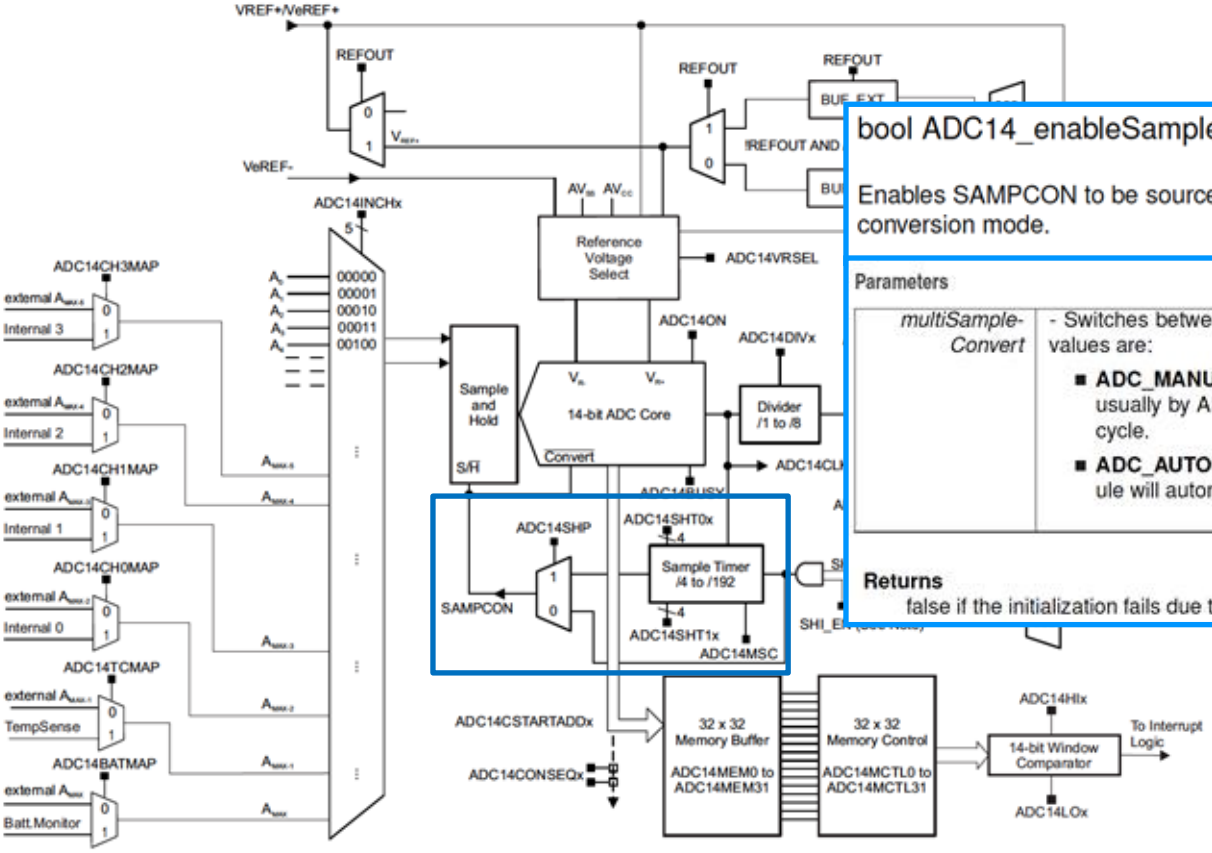
firstPulseWidth	Pulse width of the first pulse in ADCCLK cycles Possible values must be one of the following: ■ **ADC_PULSE_WIDTH_4** [DEFAULT] ■ **ADC_PULSE_WIDTH_8** ■ **ADC_PULSE_WIDTH_16** ■ **ADC_PULSE_WIDTH_32** ■ **ADC_PULSE_WIDTH_64** ■ **ADC_PULSE_WIDTH_96** ■ **ADC_PULSE_WIDTH_128** ■ **ADC_PULSE_WIDTH_192**
second-PulseWidth	Pulse width of the second pulse in ADCCLK cycles. Possible values must be one of the following: ■ **ADC_PULSE_WIDTH_4** [DEFAULT] ■ **ADC_PULSE_WIDTH_8** ■ **ADC_PULSE_WIDTH_16** ■ **ADC_PULSE_WIDTH_32** ■ **ADC_PULSE_WIDTH_64** ■ **ADC_PULSE_WIDTH_96** ■ **ADC_PULSE_WIDTH_128** ■ **ADC_PULSE_WIDTH_192**

Returns

false if setting fails due to an in progress conversion

On-chip Temperature Sensor

```
/* Initializing ADC (MCLK/1/1) with temperature sensor routed */
ADC14_enableModule();
ADC14_initModule(ADC_CLOCKSOURCE_MCLK, ADC_PREDIVIDER_1,
ADC_DIVIDER_1,ADC_TEMPSENSEMAP);
/* Configuring ADC Memory (ADC_MEM0 A22 (Temperature Sensor) in repeat mode */
ADC14_configureSingleSampleMode(ADC_MEM0, true);
ADC14_configureConversionMemory(ADC_MEM0,
ADC_VREFPOS_INTBUF_VREFNEG_VSS,ADC_INPUT_A22, false);
/* Configuring the sample/hold time for 192 */
ADC14_setSampleHoldTime(ADC_PULSE_WIDTH_192,ADC_PULSE_WIDTH_192);
/* Enabling sample timer in auto iteration mode and interrupts*/
ADC14_enableSampleTimer(ADC_AUTOMATIC_ITERATION);
```



bool ADC14_enableSampleTimer (uint32_t *multiSampleConvert*)

Enables SAMPCON to be sourced from the sampling timer and to configures multi sample and conversion mode.

Parameters

multiSample-Convert	- Switches between manual and automatic iteration when using the sample timer. Valid values are: ■ **ADC_MANUAL_ITERATION** The user will have to manually set the SHI signal (usually by ADC14_toggleConversionTrigger) at the end of each sample/conversion cycle. ■ **ADC_AUTOMATIC_ITERATION** After one sample/convert is finished, the ADC module will automatically continue on to the next sample

Returns

false if the initialization fails due to an in progress conversion

On-chip Temperature Sensor

```
void ADC14_IRQHandler(void){
    uint64_t status;
    int16_t conRes;

    status = ADC14_getEnabledInterruptStatus();
    ADC14_clearInterruptFlag(status);

    if(status & ADC_INT0){
        conRes = ((ADC14_getResult(ADC_MEM0) - cal30) * 55);
        tempC = (conRes / calDifference) + 30.0f;
        tempF = tempC * 9.0f / 5.0f + 32.0f;
    }

}
```

Lab #1: Periodic Temperature Sensing

- Implement the following FSM **usingDriverLib**.
  - Sample temperature **every 1 second** at transition to the corresponding state.

